Dynamic Programming

Minimal Grid Path

```cpp
// cses/Dynamic Programming/minimal_grid_path.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int main() {
  int n;
  cin >> n;

  vector<string> grid(n);
  for (auto &s : grid)
    cin >> s;

  string ans;
  ans += grid[0][0];

  // Current cells are represented only by their row.
  // On diagonal d, column = d - row.
  vector<int> cur = {0}, nxt;
  vector<int> seen(n, -1);

  for (int d = 0; d < 2 * n - 2; d++) {
    char best = '{'; // one after 'Z'
    nxt.clear();

    auto add = [&](int r, int c) {
      char ch = grid[r][c];

      if (ch < best) {
        best = ch;
        nxt.clear();
      }

      if (ch == best && seen[r] != d) {
        seen[r] = d;
        nxt.push_back(r);
      }
    };

    for (int r : cur) {
      int c = d - r;

      if (c + 1 < n)
        add(r, c + 1);

      if (r + 1 < n)
        add(r + 1, c);
    }

    ans += best;
    cur.swap(nxt);
  }

  cout << ans << '\n';
}
```

Graph Algorithms

Knight's Tour

```cpp
// cses/Graph Algorithms/knights_tour.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int ans[8][8];

int dx[] = {1, 2, 2, 1, -1, -2, -2, -1};
int dy[] = {-2, -1, 1, 2, 2, 1, -1, -2};

bool valid(int x, int y) {
  return x >= 0 && x < 8 && y >= 0 && y < 8 && ans[y][x] == 0;
}

// Number of moves available after entering (x, y).
int degree(int x, int y) {
  int cnt = 0;

  for (int k = 0; k < 8; k++)
    if (valid(x + dx[k], y + dy[k]))
      cnt++;

  return cnt;
}

bool dfs(int x, int y, int step) {
  ans[y][x] = step;

  if (step == 64)
    return true;

  // {degree, x, y}
  vector<tuple<int, int, int>> moves;

  for (int k = 0; k < 8; k++) {
    int nx = x + dx[k];
    int ny = y + dy[k];

    if (valid(nx, ny))
      moves.push_back({degree(nx, ny), nx, ny});
  }

  // Warnsdorff: visit the most constrained square first.
  sort(moves.begin(), moves.end());

  for (auto [d, nx, ny] : moves)
    if (dfs(nx, ny, step + 1))
      return true;

  ans[y][x] = 0;
  return false;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int x, y;
  cin >> x >> y;
  x--, y--;

  dfs(x, y, 1);

  for (int i = 0; i < 8; i++) {
    for (int j = 0; j < 8; j++)
      cout << ans[i][j] << ' ';
    cout << '\n';
  }
}
```

Range Queries

Visible Buildings Queries

```cpp
// cses/Range Queries/visible_buildings_queries.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int main() {
  int n, q;
  cin >> n >> q;

  vector<ll> h(n + 1);
  for (int i = 1; i <= n; i++)
    cin >> h[i];

  const int LOG = 18;
  vector<vector<int>> up(LOG, vector<int>(n + 2, n + 1));

  // Next strictly greater element to the right
  stack<int> st;

  for (int i = n; i >= 1; i--) {
    while (!st.empty() && h[st.top()] <= h[i])
      st.pop();

    if (!st.empty())
      up[0][i] = st.top();

    st.push(i);
  }

  // Binary lifting
  for (int k = 1; k < LOG; k++)
    for (int i = 1; i <= n; i++)
      up[k][i] = up[k - 1][up[k - 1][i]];

  while (q--) {
    int a, b;
    cin >> a >> b;

    int cur = a;
    int ans = 1; // building a is always visible

    for (int k = LOG - 1; k >= 0; k--) {
      if (up[k][cur] <= b) {
        cur = up[k][cur];
        ans += 1 << k;
      }
    }

    cout << ans << '\n';
  }
}
```

Tree Algorithms

Fixed Length Paths I

```cpp
// cses/Tree Algorithms/fixed_length_paths_I.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

const int N = 200005;

int n, k;
vector<int> g[N];

int sz[N], cnt[N];
bool dead[N];

ll ans = 0;

void getSize(int u, int p) {
  sz[u] = 1;

  for (int v : g[u])
    if (v != p && !dead[v]) {
      getSize(v, u);
      sz[u] += sz[v];
    }
}

int getCentroid(int u, int p, int total) {
  for (int v : g[u])
    if (v != p && !dead[v] && sz[v] > total / 2)
      return getCentroid(v, u, total);

  return u;
}

void getDepths(int u, int p, int d, vector<int> &depths) {
  if (d > k)
    return;

  depths.push_back(d);

  for (int v : g[u])
    if (v != p && !dead[v])
      getDepths(v, u, d + 1, depths);
}

void decompose(int start) {
  getSize(start, 0);

  int c = getCentroid(start, 0, sz[start]);
  dead[c] = true;

  // cnt[d] = nodes at distance d from c
  // in previously processed subtrees.
  cnt[0] = 1;

  int maxDepth = 0;
  vector<int> depths;

  for (int v : g[c]) {
    if (dead[v])
      continue;

    depths.clear();
    getDepths(v, c, 1, depths);

    // Count paths whose two endpoints are in
    // different subtrees (or one endpoint is c).
    for (int d : depths)
      ans += cnt[k - d];

    // Add this subtree only after counting it.
    for (int d : depths) {
      cnt[d]++;
      maxDepth = max(maxDepth, d);
    }
  }

  // Restore cnt[] to zero for the next centroid.
  for (int d = 0; d <= maxDepth; d++)
    cnt[d] = 0;

  for (int v : g[c])
    if (!dead[v])
      decompose(v);
}

int main() {
  cin >> n >> k;

  for (int i = 1; i < n; i++) {
    int a, b;
    cin >> a >> b;

    g[a].push_back(b);
    g[b].push_back(a);
  }

  decompose(1);

  cout << ans << '\n';
}
```

Fixed Length Paths II

```cpp
// cses/Tree Algorithms/fixed_length_paths_II.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

const int N = 200005;

int n, k1, k2;
vector<int> g[N];

int sz[N], par[N];
bool dead[N];

int suf[N], cur[N];
ll ans = 0;

// Reused buffers to avoid lots of allocations.
vector<int> order_, st_;
vector<array<int, 3>> walk_;

int get_centroid(int start) {
  order_.clear();
  st_.clear();

  st_.push_back(start);
  par[start] = 0;

  // Get all nodes in this component.
  while (!st_.empty()) {
    int u = st_.back();
    st_.pop_back();

    order_.push_back(u);

    for (int v : g[u])
      if (v != par[u] && !dead[v]) {
        par[v] = u;
        st_.push_back(v);
      }
  }

  // Subtree sizes.
  for (int i = (int)order_.size() - 1; i >= 0; i--) {
    int u = order_[i];
    sz[u] = 1;

    for (int v : g[u])
      if (!dead[v] && par[v] == u)
        sz[u] += sz[v];
  }

  int total = sz[start];

  // Find a node whose largest remaining component
  // has size <= total / 2.
  for (int u : order_) {
    int largest = total - sz[u];

    for (int v : g[u])
      if (!dead[v] && par[v] == u)
        largest = max(largest, sz[v]);

    if (largest <= total / 2)
      return u;
  }

  return start;
}

// Process one child subtree of the centroid.
//
// suf[d] = number of already processed nodes
//          whose depth from the centroid is >= d.
int process_subtree(int root, int centroid) {
  walk_.clear();
  walk_.push_back({root, centroid, 1});

  int mx = 0;

  while (!walk_.empty()) {
    auto [u, p, d] = walk_.back();
    walk_.pop_back();

    if (d > k2)
      continue;

    // Need another node at depth x such that:
    // k1 <= d + x <= k2.
    int lo = max(0, k1 - d);
    int hi = k2 - d;

    ans += suf[lo] - suf[hi + 1];

    cur[d]++;
    mx = max(mx, d);

    for (int v : g[u])
      if (v != p && !dead[v])
        walk_.push_back({v, u, d + 1});
  }

  // Add this subtree to suf.
  int sum = 0;

  for (int d = mx; d >= 0; d--) {
    sum += cur[d];
    suf[d] += sum;
    cur[d] = 0;
  }

  return mx;
}

void decompose(int start) {
  int c = get_centroid(start);
  dead[c] = true;

  // Centroid itself has depth 0.
  suf[0] = 1;

  int max_depth = 0;

  for (int v : g[c])
    if (!dead[v])
      max_depth = max(max_depth, process_subtree(v, c));

  // Clear suffix counts for the next centroid.
  for (int d = 0; d <= max_depth; d++)
    suf[d] = 0;
```

```
  for (int v : g[c])
    if (!dead[v])
      decompose(v);
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  cin >> n >> k1 >> k2;

  for (int i = 0; i < n - 1; i++) {
    int a, b;
    cin >> a >> b;
    g[a].push_back(b);
    g[b].push_back(a);
  }

  decompose(1);

  cout << ans << '\n';
}
```

Mathematics

System of Linear Equations

```cpp
// cses/Mathematics/system_of_linear_equations.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

const ll MOD = 1e9 + 7;

ll modpow(ll a, ll e) {
  ll r = 1;
  while (e) {
    if (e & 1)
      r = r * a % MOD;
    a = a * a % MOD;
    e >>= 1;
  }
  return r;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<vector<ll>> a(n, vector<ll>(m + 1));

  for (auto &row : a)
    for (ll &x : row)
      cin >> x;

  vector<int> where(m, -1);

  int row = 0;

  // Gaussian elimination.
  for (int col = 0; col < m && row < n; col++) {
    int sel = row;

    while (sel < n && a[sel][col] == 0)
      sel++;

    if (sel == n)
      continue;

    swap(a[sel], a[row]);
    where[col] = row;

    ll inv = modpow(a[row][col], MOD - 2);

    for (int j = col; j <= m; j++)
      a[row][j] = a[row][j] * inv % MOD;

    for (int i = row + 1; i < n; i++) {
      if (a[i][col] == 0)
        continue;

      ll f = a[i][col];

      for (int j = col; j <= m; j++) {
        a[i][j] -= f * a[row][j] % MOD;
        if (a[i][j] < 0)
          a[i][j] += MOD;
      }
    }

    row++;
  }

  // Check for 0 = nonzero.
  for (int i = row; i < n; i++) {
    bool zero = true;

    for (int j = 0; j < m; j++)
      if (a[i][j] != 0)
        zero = false;

    if (zero && a[i][m] != 0) {
      cout << -1 << '\n';
      return 0;
    }
  }

  // Free variables stay 0.
  vector<ll> x(m);

  // Back substitution.
  for (int col = m - 1; col >= 0; col--) {
    if (where[col] == -1)
      continue;

    int r = where[col];
    x[col] = a[r][m];

    for (int j = col + 1; j < m; j++) {
      x[col] -= a[r][j] * x[j] % MOD;
      if (x[col] < 0)
        x[col] += MOD;
    }
  }

  for (int i = 0; i < m; i++)
    cout << x[i] << " \n"[i == m - 1];
}
```

String Algorithms

Inverse Suffix Array

```cpp
// cses/String Algorithms/inverse_suffix_array.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<int> sa(n), rank(n);

  for (int &x : sa) {
    cin >> x;
    --x;
  }

  for (int i = 0; i < n; i++)
    rank[sa[i]] = i;

  string s(n, 'a');
  int c = 0;

  for (int i = 1; i < n; i++) {
    int a = sa[i - 1];
    int b = sa[i];

    // Rank of the suffix after removing the first character.
    // The empty suffix is smaller than every nonempty suffix.
    int ra = (a + 1 == n ? -1 : rank[a + 1]);
    int rb = (b + 1 == n ? -1 : rank[b + 1]);

    // Same first character would give the wrong order.
    if (ra > rb)
      c++;

    if (c >= 26) {
      cout << -1 << '\n';
      return 0;
    }

    s[b] = 'a' + c;
  }

  cout << s << '\n';
}
```

Geometry

Robot Path

```cpp
// cses/Geometry/robot_path.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

const ll INF = (1LL << 62);

struct Seg {
  ll x1, y1, x2, y2;
};

vector<Seg> seg;
vector<char> dir;
vector<ll> len;

// Checks intersections between parallel segments.
bool parallel_intersection(int m, bool vertical) {
  vector<array<ll, 3>> v;

  for (int i = 0; i < m; i++) {
    auto s = seg[i];

    if (vertical && s.x1 == s.x2)
      v.push_back({s.x1, min(s.y1, s.y2), max(s.y1, s.y2)});

    if (!vertical && s.y1 == s.y2)
      v.push_back({s.y1, min(s.x1, s.x2), max(s.x1, s.x2)});
  }

  sort(v.begin(), v.end());

  for (int i = 1; i < (int)v.size(); i++)
    if (v[i][0] == v[i - 1][0] && v[i][1] <= v[i - 1][2])
      return true;

  return false;
}

// Checks horizontal/vertical intersections with a sweep line.
bool perpendicular_intersection(int m) {
  // {x, type, y1, y2}
  // type: 0 = add horizontal
  //       1 = query vertical
  //       2 = remove horizontal
  vector<array<ll, 4>> ev;

  for (int i = 0; i < m; i++) {
    auto s = seg[i];

    if (s.y1 == s.y2) {
      ll l = min(s.x1, s.x2);
      ll r = max(s.x1, s.x2);

      ev.push_back({l, 0, s.y1, 0});
      ev.push_back({r, 2, s.y1, 0});
    } else {
      ev.push_back({s.x1, 1, min(s.y1, s.y2), max(s.y1, s.y2)});
    }
  }

  sort(ev.begin(), ev.end());

  multiset<ll> active;

  for (auto [x, type, a, b] : ev) {
    if (type == 0) {
      active.insert(a);
    } else if (type == 2) {
      active.erase(active.find(a));
    } else {
      auto it = active.lower_bound(a);

      if (it != active.end() && *it <= b)
        return true;
    }
  }

  return false;
}

bool has_intersection(int m) {
  return parallel_intersection(m, true) || parallel_intersection(m, false) ||
         perpendicular_intersection(m);
}

// Distance from a.x1,a.y1 to the first intersection with b.
ll intersection_dist(const Seg &a, const Seg &b) {
  bool ah = (a.y1 == a.y2);
  bool bh = (b.y1 == b.y2);

  // horizontal-horizontal
  if (ah && bh) {
    if (a.y1 != b.y1)
      return INF;

    ll l = max(min(a.x1, a.x2), min(b.x1, b.x2));

    ll r = min(max(a.x1, a.x2), max(b.x1, b.x2));

    if (l > r)
      return INF;

    if (a.x2 >= a.x1)
      return l - a.x1;

    return a.x1 - r;
  }

  // vertical-vertical
  if (!ah && !bh) {
    if (a.x1 != b.x1)
      return INF;

    ll l = max(min(a.y1, a.y2), min(b.y1, b.y2));

    ll r = min(max(a.y1, a.y2), max(b.y1, b.y2));

    if (l > r)
      return INF;

    if (a.y2 >= a.y1)
      return l - a.y1;

    return a.y1 - r;
  }

  // a horizontal, b vertical
  if (ah) {
    ll x = b.x1;
    ll y = a.y1;

    if (x < min(a.x1, a.x2) || x > max(a.x1, a.x2) || y < min(b.y1, b.y2) ||
        y > max(b.y1, b.y2))
      return INF;

    return abs(x - a.x1);
  }

  // a vertical, b horizontal
  ll x = a.x1;
  ll y = b.y1;

  if (x < min(b.x1, b.x2) || x > max(b.x1, b.x2) || y < min(a.y1, a.y2) ||
      y > max(a.y1, a.y2))
    return INF;

  return abs(y - a.y1);
}

bool opposite(char a, char b) {
  return (a == 'U' && b == 'D') || (a == 'D' && b == 'U') ||
         (a == 'L' && b == 'R') || (a == 'R' && b == 'L');
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  seg.resize(n);
  dir.resize(n);
  len.resize(n);

  ll x = 0, y = 0;

  for (int i = 0; i < n; i++) {
    char d;
    ll z;
    cin >> d >> z;

    ll dx = (d == 'R') - (d == 'L');
    ll dy = (d == 'U') - (d == 'D');

    // For i > 0, exclude the point where the
    // previous command ended.
    seg[i] = {x + (i > 0) * dx, y + (i > 0) * dy, x + z * dx, y + z * dy};

    x += z * dx;
    y += z * dy;

    dir[i] = d;
    len[i] = z;
  }

  // First command that creates an intersection.
  int lo = 0, hi = n;

  while (lo < hi) {
    int mid = (lo + hi) / 2;

    if (has_intersection(mid + 1))
      hi = mid;
    else
      lo = mid + 1;
  }

  int bad = lo;

  ll ans = 0;

  for (int i = 0; i < bad; i++)
    ans += len[i];

  if (bad == n) {
    cout << ans << '\n';
    return 0;
  }

  ll d = INF;

  for (int i = 0; i < bad; i++)
    d = min(d, intersection_dist(seg[bad], seg[i]));

  // seg[bad] begins one unit after the real command start.
  ans += 1 + d;

  // Reversing direction overlaps the previous segment immediately.
  if (opposite(dir[bad], dir[bad - 1]))
    ans--;

  cout << ans << '\n';
}
```

Advanced Techniques

Corner Subgrid Check

```cpp
// cses/Advanced Techniques/corner_subgrid_check.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<vector<vector<int>>> pos(n, vector<vector<int>>(k));

  for (int i = 0; i < n; i++) {
    string s;
    cin >> s;

    for (int j = 0; j < n; j++)
      pos[i][s[j] - 'A'].push_back(j);
  }

  // seen[a * n + b] = letter for which pair (a, b) was seen.
  vector<unsigned char> seen(n * n);

  ll maxPairs = 1LL * n * (n - 1) / 2;

  for (int c = 0; c < k; c++) {
    ll pairs = 0;

    for (int i = 0; i < n; i++) {
      ll sz = pos[i][c].size();
      pairs += sz * (sz - 1) / 2;
    }

    // More occurrences of column-pairs than distinct column-pairs:
    // some pair must appear in two rows.
    if (pairs > maxPairs) {
      cout << "YES\n";
      continue;
    }

    bool ok = false;
    unsigned char mark = c + 1;

    for (int i = 0; i < n && !ok; i++) {
      auto &v = pos[i][c];

      for (int a = 0; a < (int)v.size() && !ok; a++)
        for (int b = a + 1; b < (int)v.size(); b++) {
          int id = v[a] * n + v[b];

          if (seen[id] == mark) {
            ok = true;
            break;
          }

          seen[id] = mark;
        }
    }

    cout << (ok ? "YES\n" : "NO\n");
  }
}
```

Sliding Window Problems

Sliding Window Advertisement

```cpp
// cses/Sliding Window Problems/sliding_window_advertisement.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

struct LiChao {
  struct Line {
    ll m = 0, b = 0;

    ll get(ll x) const { return m * x + b; }
  };

  int n;
  vector<Line> tree;

  LiChao(int n) : n(n), tree(4 * n) {}

  void add_line(Line nw, int p, int l, int r) {
    int mid = (l + r) / 2;

    bool lef = nw.get(l) > tree[p].get(l);
    bool mid_better = nw.get(mid) > tree[p].get(mid);

    if (mid_better)
      swap(nw, tree[p]);

    if (l == r)
      return;

    if (lef != mid_better)
      add_line(nw, 2 * p, l, mid);
    else
      add_line(nw, 2 * p + 1, mid + 1, r);
  }

  void add_segment(Line nw, int ql, int qr, int p, int l, int r) {
    if (qr < l || r < ql)
      return;

    if (ql <= l && r <= qr) {
      add_line(nw, p, l, r);
      return;
    }

    int mid = (l + r) / 2;
    add_segment(nw, ql, qr, 2 * p, l, mid);
    add_segment(nw, ql, qr, 2 * p + 1, mid + 1, r);
  }

  void add_segment(Line nw, int l, int r) {
    l = max(l, 0);
    r = min(r, n - 1);

    if (l <= r)
      add_segment(nw, l, r, 1, 0, n - 1);
  }

  ll query(int x, int p, int l, int r) {
    ll ans = tree[p].get(x);

    if (l == r)
      return ans;

    int mid = (l + r) / 2;

    if (x <= mid)
      return max(ans, query(x, 2 * p, l, mid));
    else
      return max(ans, query(x, 2 * p + 1, mid + 1, r));
  }

  ll query(int x) { return query(x, 1, 0, n - 1); }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<ll> a(n);
  for (ll &x : a)
    cin >> x;

  // Maximal interval [prv[i] + 1, nxt[i] - 1]
  // where a[i] can be the minimum.
  vector<int> prv(n), nxt(n), st;

  for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.back()] >= a[i])
      st.pop_back();
    prv[i] = st.empty() ? -1 : st.back();
    st.push_back(i);
  }

  st.clear();

  for (int i = n - 1; i >= 0; i--) {
    while (!st.empty() && a[st.back()] >= a[i])
      st.pop_back();

    nxt[i] = st.empty() ? n : st.back();
    st.push_back(i);
  }

  int windows = n - k + 1;
  LiChao cht(windows);

  for (int i = 0; i < n; i++) {
    int l = prv[i] + 1;
    int r = nxt[i] - 1;
    int len = r - l + 1;
    ll h = a[i];

    if (len >= k) {
      // Increasing part.
      cht.add_segment({h, h * (k - l)}, l - k + 1, l);

      // Whole window fits.
      cht.add_segment({0, h * k}, l, r - k + 1);

      // Decreasing part.
      cht.add_segment({-h, h * (r + 1)}, r - k + 1, r);
    } else {
      // Window enters [l, r].
      cht.add_segment({h, h * (k - l)}, l - k + 1, r - k + 1);

      // Window completely covers [l, r].
      cht.add_segment({0, h * len}, r - k + 1, l);

      // Window leaves [l, r].
      cht.add_segment({-h, h * (r + 1)}, l, r);
    }
  }

  for (int i = 0; i < windows; i++)
    cout << cht.query(i) << " \n"[i + 1 == windows];
}
```

Interactive Problems

Colored Chairs

```cpp
// cses/Interactive Problems/colored_chairs.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  int n;
  cin >> n;

  auto ask = [&](int i) {
    cout << "? " << i << endl;
    char c;
    cin >> c;
    return c;
  };

  int l = 1, r = n;
  char cl = ask(l);
  char cr = ask(r);

  // Since n is odd, chairs n and 1 are adjacent.
  if (cl == cr) {
    cout << "! " << n << endl;
    return 0;
  }

  while (r - l > 1) {
    int m = (l + r) / 2;
    char cm = ask(m);

    // With perfect alternation:
    // even distance -> same color
    // odd distance  -> different colors
    bool left_bad = (cl == cm) != ((m - l) % 2 == 0);

    if (left_bad) {
      r = m;
      cr = cm;
    } else {
      l = m;
      cl = cm;
    }
  }

  cout << "! " << l << endl;
}
```

Hidden Integer

```cpp
// cses/Interactive Problems/hidden_integer.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ll l = 1, r = 1e9;

  while (l < r) {
    ll mid = (l + r) / 2;

    cout << "? " << mid << endl; // endl flushes output

    string s;
    cin >> s;

    if (s == "YES")
      l = mid + 1;
    else
      r = mid;
  }

  cout << "! " << l << endl;
}
```

Hidden Permutation

```cpp
// cses/Interactive Problems/hidden_permutation.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

bool ask(int i, int j) {
  cout << "? " << i << ' ' << j << endl;

  string s;
  cin >> s;
  return s == "YES";
}

void merge_sort(vector<int> &v, int l, int r) {
  if (r - l <= 1)
    return;

  int m = (l + r) / 2;
  merge_sort(v, l, m);
  merge_sort(v, m, r);

  vector<int> tmp;
  int i = l, j = m;

  while (i < m && j < r) {
    if (ask(v[i], v[j]))
      tmp.push_back(v[i++]);
    else
      tmp.push_back(v[j++]);
  }

  while (i < m)
    tmp.push_back(v[i++]);
  while (j < r)
    tmp.push_back(v[j++]);

  for (int k = 0; k < (int)tmp.size(); k++)
    v[l + k] = tmp[k];
}

int main() {
  int n;
  cin >> n;

  vector<int> p(n);
  iota(p.begin(), p.end(), 1);

  merge_sort(p, 0, n);

  vector<int> ans(n);

  for (int i = 0; i < n; i++)
    ans[p[i] - 1] = i + 1;

  cout << "!";
  for (int x : ans)
    cout << ' ' << x;
  cout << endl;
}
```

Inversion Sorting

```cpp
// cses/Interactive Problems/inversion_sorting.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  auto ask = [&](int l, int r) {
    cout << l << " " << r << endl;

    ll x;
    cin >> x;

    if (x < 0)
      exit(0);

    return x;
  };

  vector<ll> pref(n + 1);

  // Reverse everything once so we learn the inversion count
  // of the permutation we will reconstruct.
  ll base = ask(1, n);

  if (base == 0)
    return 0;

  pref[1] = 0;
  pref[n] = base;

  // Recover inversion count of every prefix.
  for (int k = 2; k < n; k++) {
    ll after = ask(1, k);

    if (after == 0)
      return 0;

    ll pairs = 1LL * k * (k - 1) / 2;

    // after - base = pairs - 2 * pref[k]
    pref[k] = (pairs - (after - base)) / 2;

    // Restore the permutation.
    base = ask(1, k);

    if (base == 0)
      return 0;
  }

  // e[i] = number of previous elements greater than a[i].
  // This is an inversion sequence, so reconstruct the permutation.
  vector<int> remaining(n);
  iota(remaining.begin(), remaining.end(), 1);

  vector<int> a(n + 1);

  for (int i = n; i >= 1; i--) {
    int e = pref[i] - pref[i - 1];
    int rank = i - e; // rank of a[i] among remaining values

    a[i] = remaining[rank - 1];
    remaining.erase(remaining.begin() + rank - 1);
  }

  // Now we know the permutation. Put i into position i.
  for (int i = 1; i <= n; i++) {
    if (a[i] == i)
      continue;

    int j = i;
    while (a[j] != i)
      j++;

    ll inv = ask(i, j);

    if (inv == 0)
      return 0;

    reverse(a.begin() + i, a.begin() + j + 1);
  }
}
```

K-th Highest Score

```cpp
// cses/Interactive Problems/k_th_highest_score.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll INF = LLONG_MAX / 4;

int n, k;

ll ask(char c, int i) {
  if (i == 0)
    return INF;
```

```
  if (i == n + 1)
    return -INF;

  cout << c << " " << i << endl;

  ll x;
  cin >> x;
  return x;
}

int main() {
  cin >> n >> k;

  // x = number of Finnish scores among the top k.
  int l = max(0, k - n);
  int r = min(k, n);

  // Find the largest x such that
  // F[x] > S[k-x+1].
  while (l < r) {
    int x = (l + r + 1) / 2;
    int y = k - x;

    if (ask('F', x) > ask('S', y + 1))
      l = x;
    else
      r = x - 1;
  }

  int x = l;
  int y = k - x;

  // The k-th highest is the smaller of the
  // last selected score from each country.
  ll ans = min(ask('F', x), ask('S', y));

  cout << "! " << ans << endl;
}
```

Permuted Binary Strings

```
// cses/Interactive Problems/permuted_binary_strings.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  int n;
  cin >> n;

  int q = 0;
  while ((1 << q) < n)
    q++;

  vector<int> ans(n);

  for (int bit = 0; bit < q; bit++) {
    string s(n, '0');

    for (int i = 0; i < n; i++)
      if ((i >> bit) & 1)
        s[i] = '1';

    cout << "? " << s << endl;

    string res;
    cin >> res;

    for (int i = 0; i < n; i++)
      if (res[i] == '1')
        ans[i] |= 1 << bit;
  }

  cout << "!";
  for (int x : ans)
    cout << " " << x + 1;
  cout << endl;
}
```

Bitwise Operations

All Subarray XORs

```cpp
// cses/Bitwise Operations/all_subarray_xors.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

void fwht(vector<ll> &a) {
  int n = a.size();

  for (int len = 1; len < n; len <<= 1)
    for (int i = 0; i < n; i += 2 * len)
      for (int j = 0; j < len; j++) {
        ll x = a[i + j];
        ll y = a[i + j + len];

        a[i + j] = x + y;
        a[i + j + len] = x - y;
      }
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  const int M = 1 << 20;
  vector<ll> f(M);

  int pref = 0;
  f[0]++;

  for (int i = 0; i < n; i++) {
    int x;
    cin >> x;

    pref ^= x;
    f[pref]++;
  }

  // XOR convolution f * f.
  fwht(f);

  for (ll &x : f)
    x *= x;

  fwht(f);

  for (ll &x : f)
    x /= M;

  vector<int> ans;

  for (int x = 0; x < M; x++) {
    if (x == 0) {
      // f[0] also counts pairing every prefix with itself.
      if (f[x] > n + 1)
        ans.push_back(x);
    } else if (f[x] > 0) {
      ans.push_back(x);
    }
  }

  cout << ans.size() << '\n';

  for (int x : ans)
    cout << x << ' ';

  cout << '\n';
}
```

AND Subset Count

```cpp
// cses/Bitwise Operations/and_subset_count.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  int m = 1;
  while (m <= n)
    m <<= 1;

  vector<int> cnt(m);

  for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    cnt[x]++;
  }

  // cnt[mask] = number of elements containing all bits of mask
  for (int bit = 1; bit < m; bit <<= 1)
    for (int mask = 0; mask < m; mask++)
      if (!(mask & bit))
        cnt[mask] += cnt[mask | bit];

  vector<ll> pow2(n + 1, 1);
  for (int i = 1; i <= n; i++)
    pow2[i] = 2 * pow2[i - 1] % MOD;

  // ans[mask] = subsets whose AND contains mask
  vector<ll> ans(m);
  for (int mask = 0; mask < m; mask++)
    ans[mask] = (pow2[cnt[mask]] - 1 + MOD) % MOD;

  // Möbius inversion: containing mask -> exactly mask
  for (int bit = 1; bit < m; bit <<= 1)
    for (int mask = 0; mask < m; mask++)
      if (!(mask & bit)) {
        ans[mask] -= ans[mask | bit];
        if (ans[mask] < 0)
          ans[mask] += MOD;
      }

  for (int k = 0; k <= n; k++)
    cout << ans[k] << " \n"[k == n];
}
```

K Subset XORs

```cpp
// cses/Bitwise Operations/k_subset_xors.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  const int B = 30;
  ll basis[B] = {};

  // Build xor basis, pivot = highest set bit.
  for (int t = 0; t < n; t++) {
    ll x;
    cin >> x;

    for (int i = B - 1; i >= 0; i--) {
      if (!(x >> i & 1))
        continue;

      if (basis[i])
        x ^= basis[i];
      else {
        basis[i] = x;
        break;
      }
    }
  }

  // Reduced basis:
  // basis[j] must not contain the pivot bit of basis[i], i < j.
  //
  // After this, masks 0,1,2,... produce xor values
  // directly in increasing order.
  for (int i = 0; i < B; i++)
    if (basis[i])
      for (int j = i + 1; j < B; j++)
        if (basis[j] >> i & 1)
          basis[j] ^= basis[i];

  vector<ll> b;
  for (int i = 0; i < B; i++)
    if (basis[i])
      b.push_back(basis[i]);

  int rank = b.size();

  // Every distinct xor appears 2^(n-rank) times.
  ll copies = 1;
  for (int i = 0; i < n - rank && copies < k; i++)
    copies = min<ll>(k, copies * 2);

  int need = (k + copies - 1) / copies;

  int printed = 0;

  for (int mask = 0; mask < need; mask++) {
    ll x = 0;

    for (int i = 0; i < rank; i++)
      if (mask >> i & 1)
        x ^= b[i];

    for (ll rep = 0; rep < copies && printed < k; rep++) {
      if (printed++)
        cout << ' ';
      cout << x;
    }
  }

  cout << '\n';
}
```

SOS Bit Problem

```cpp
// cses/Bitwise Operations/sos_bit_problem.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  const int B = 20;
  const int M = 1 << B;

  vector<int> a(n), sub(M), sup(M);

  for (int &x : a) {
    cin >> x;
    sub[x]++;
    sup[x]++;
  }

  // sub[x] = number of y such that y is a submask of x
  //        -> x | y = x
  for (int b = 0; b < B; b++)
    for (int mask = 0; mask < M; mask++)
      if (mask & (1 << b))
        sub[mask] += sub[mask ^ (1 << b)];
```

```
// sup[x] = number of y such that y is a supermask of x
//        -> x & y = x
for (int b = 0; b < B; b++)
  for (int mask = 0; mask < M; mask++)
    if (!(mask & (1 << b)))
      sup[mask] += sup[mask ^ (1 << b)];

int all = M - 1;

for (int x : a) {
  // x & y == 0 iff y is a submask of ~x.
  int disjoint = sub[all ^ x];

  cout << sub[x] << ' ' << sup[x] << ' ' << n - disjoint << '\n';
}
```

```
}
```

XOR Pyramid Row

```
// cses/Bitwise Operations/xor_pyramid_row.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<ll> a(n);
  for (ll &x : a)
    cin >> x;

  int m = n - k;

  for (int d = 1; d <= m; d <<= 1)
    if (m & d)
      for (int i = 0; i + d < n; i++)
        a[i] ^= a[i + d];

  for (int i = 0; i < k; i++)
    cout << a[i] << " \n"[i == k - 1];
}
```

Construction Problems

Distinct Sums Grid

```cpp
// cses/Construction Problems/distinct_sums_grid.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  if (n <= 3) {
    cout << "IMPOSSIBLE\n";
    return 0;
  }

  if (n == 4) {
    cout << "3 4 4 3\n";
    cout << "2 3 4 4\n";
    cout << "1 2 2 3\n";
    cout << "1 1 1 2\n";
    return 0;
  }

  vector<vector<int>> a(n, vector<int>(n));

  for (int i = 0; i < n; i++) {
    a[i][0] = n;

    for (int j = 1; j < n; j++)
      a[i][j] = (j > i ? i + 1 : i);
  }

  map<ll, int> row;

  for (int i = 0; i < n; i++) {
    ll sum = 0;
    for (int x : a[i])
      sum += x;
    row[sum] = i;
  }

  for (int j = 0; j < n; j++) {
    ll sum = 0;
    for (int i = 0; i < n; i++)
      sum += a[i][j];

    if (!row.count(sum))
      continue;

    int i = row[sum];

    swap(a[1][0], a[1][j]);
    swap(a[i + 1][n - 1], a[i + 2][n - 1]);
    break;
  }

  for (auto &row : a) {
    for (int x : row)
      cout << x << ' ';
    cout << '\n';
  }
}
```

Filling Trominos

```cpp
// cses/Construction Problems/filling_trominos.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int T;
  cin >> T;

  while (T--) {
    int N, M;
    cin >> N >> M;

    int n = N, m = M;
    bool transposed = false;

    // Make n divisible by 3.
    if (n % 3 != 0) {
      swap(n, m);
      transposed = true;
    }

    if (n % 3 != 0 || n == 1 || m == 1) {
      cout << "NO\n";
      continue;
    }

    // Odd 3 x odd is impossible.
    if (n % 2 && m % 2 && (n == 3 || m == 3)) {
      cout << "NO\n";
      continue;
    }

    vector<vector<int>> a(n, vector<int>(m, -1));
    int ids = 0;

    // Fill a 3 x 2 block.
    auto tile32 = [&](int r, int c) {
      int x = ids++;
      a[r][c] = a[r][c + 1] = a[r + 1][c] = x;

      x = ids++;
      a[r + 1][c + 1] = a[r + 2][c] = a[r + 2][c + 1] = x;
    };

    // Fill a 2 x 3 block.
    auto tile23 = [&](int r, int c) {
      int x = ids++;
      a[r][c] = a[r][c + 1] = a[r + 1][c] = x;

      x = ids++;
      a[r][c + 2] = a[r + 1][c + 1] = a[r + 1][c + 2] = x;
    };

    if (m % 2 == 0) {
      // Entire grid = 3 x 2 blocks.
      for (int i = 0; i < n; i += 3)
        for (int j = 0; j < m; j += 2)
          tile32(i, j);

    } else if (n % 2 == 0) {
      // First 3 columns = 2 x 3 blocks.
      for (int i = 0; i < n; i += 2)
        tile23(i, 0);

      // Rest = 3 x 2 blocks.
      for (int i = 0; i < n; i += 3)
        for (int j = 3; j < m; j += 2)
          tile32(i, j);

    } else {
      // Both dimensions are odd.
      // Use one fixed 9 x 5 solution.
      const vector<string> p = {
          "AACAA", "AECCA", "GEEGG", "GGBAG", "DBBAA",
          "DDIIG", "AAIGG", "ADFFD", "DDFDD",
      };

      bool vis[9][5] = {};

      for (int si = 0; si < 9; si++) {
        for (int sj = 0; sj < 5; sj++) {
          if (vis[si][sj])
            continue;

          int x = ids++;
          char ch = p[si][sj];

          queue<pair<int, int>> q;
          q.push({si, sj});
          vis[si][sj] = true;

          while (!q.empty()) {
            auto [i, j] = q.front();
            q.pop();

            a[i][j] = x;

            const int di[] = {1, -1, 0, 0};
            const int dj[] = {0, 0, 1, -1};

            for (int d = 0; d < 4; d++) {
              int ni = i + di[d];
              int nj = j + dj[d];

              if (ni < 0 || ni >= 9 || nj < 0 || nj >= 5)
                continue;
              if (vis[ni][nj] || p[ni][nj] != ch)
                continue;

              vis[ni][nj] = true;
              q.push({ni, nj});
            }
          }
        }
      }

      // Remaining bottom-left rectangle: (n-9) x 5.
      for (int i = 9; i < n; i += 2)
        tile23(i, 0);

      for (int i = 9; i < n; i += 3)
        tile32(i, 3);

      // Remaining right rectangle: n x (m-5).
      for (int i = 0; i < n; i += 3)
        for (int j = 5; j < m; j += 2)
          tile32(i, j);
    }

    // Build adjacency graph between trominos.
    vector<vector<int>> adj(ids);

    auto add_edge = [&](int x, int y) {
      if (x != y) {
        adj[x].push_back(y);
        adj[y].push_back(x);
      }
    };

    for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++) {
        if (i + 1 < n)
          add_edge(a[i][j], a[i + 1][j]);
        if (j + 1 < m)
          add_edge(a[i][j], a[i][j + 1]);
      }

    // Give adjacent trominos different letters.
    vector<char> color(ids);

    for (int x = 0; x < ids; x++) {
      bool used[26] = {};

      for (int y : adj[x])
        if (y < x)
          used[color[y] - 'A'] = true;

      for (int c = 0; c < 26; c++)
        if (!used[c]) {
          color[x] = 'A' + c;
          break;
        }
    }

    cout << "YES\n";
```

```
    for (int i = 0; i < N; i++) {
      for (int j = 0; j < M; j++) {
        int x = transposed ? a[j][i] : a[i][j];
        cout << color[x];
      }
      cout << '\n';
    }
  }
}
```

Grid Path Construction

```
// cses/Construction Problems/grid_path_construction.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int dy[4] = {1, 0, -1, 0};
const int dx[4] = {0, 1, 0, -1};
const char dc[4] = {'D', 'R', 'U', 'L'};

int id(char c) {
  if (c == 'D')
    return 0;
  if (c == 'R')
    return 1;
  if (c == 'U')
    return 2;
  return 3;
}

bool possible(int n, int m, int sy, int sx, int ty, int tx) {
  if (sy == ty && sx == tx)
    return n * m == 1;

  if (n > m) {
    swap(n, m);
    swap(sy, sx);
    swap(ty, tx);
  }

  if (sx > tx) {
    swap(sx, tx);
    swap(sy, ty);
  }

  if (n == 1)
    return sx == 0 && tx == m - 1;

  if (n == 2) {
    if (sx == tx)
      return sx == 0 || sx == m - 1;

    return (sx + sy) % 2 != (tx + ty) % 2;
  }

  if (n * m % 2)
    return (sx + sy) % 2 == 0 && (tx + ty) % 2 == 0;

  if (n == 3) {
    if ((sx + sy) % 2 == (tx + ty) % 2)
      return false;

    if ((sx + sy) % 2 == 1 && (sx < tx - 1 || (sy == 1 && sx < tx)))
      return false;

    if ((tx + ty) % 2 == 1 && (tx < sx - 1 || (ty == 1 && tx < sx)))
      return false;

    return true;
  }

  return (sx + sy) % 2 != (tx + ty) % 2;
}

// Exact solver only for rectangles up to 4 x 5.
string small_path(int n, int m, int sy, int sx, int ty, int tx) {
  int N = n * m;
  int S = sy * m + sx;
  int T = ty * m + tx;

  vector<char> vis(N);
  vector<int> path;
  vis[S] = true;

auto inside = [&](int y, int x) {
  return 0 <= y && y < n && 0 <= x && x < m;
};

function<bool(int, int)> dfs = [&](int v, int used) {
  if (used == N)
    return v == T;

  int y = v / m;
  int x = v % m;

  vector<pair<int, int>> cand;

  for (int d = 0; d < 4; d++) {
    int ny = y + dy[d];
    int nx = x + dx[d];

    if (!inside(ny, nx))
      continue;

    int u = ny * m + nx;

    if (vis[u])
      continue;

    if (u == T && used + 1 < N)
      continue;

    int deg = 0;

    for (int e = 0; e < 4; e++) {
      int yy = ny + dy[e];
      int xx = nx + dx[e];

      if (inside(yy, xx) && !vis[yy * m + xx])
        deg++;
    }

    cand.push_back({deg, d});
  }

  sort(cand.begin(), cand.end());

  for (auto [_, d] : cand) {
    int ny = y + dy[d];
    int nx = x + dx[d];
    int u = ny * m + nx;

    vis[u] = true;
    path.push_back(d);

    bool connected = true;

    if (used + 1 < N) {
      vector<char> seen(N);
      queue<int> q;

      q.push(u);
      seen[u] = true;

      int cnt = 1;

      while (!q.empty()) {
        int a = q.front();
        q.pop();

        int ay = a / m;
        int ax = a % m;

        for (int e = 0; e < 4; e++) {
          int by = ay + dy[e];
          int bx = ax + dx[e];

          if (!inside(by, bx))
            continue;

          int b = by * m + bx;

          if (seen[b] || (vis[b] && b != u))
            continue;

          seen[b] = true;
          q.push(b);
          cnt++;
        }
      }

      connected = (cnt == N - used);
    }

    if (connected && dfs(u, used + 1))
      return true;

    path.pop_back();
    vis[u] = false;
  }

  return false;
};

dfs(S, 1);

string res;
for (int d : path)
  res += dc[d];

return res;
}

string build(int n, int m, int sy, int sx, int ty, int tx) {
  // Rotate until n <= m.
  if (n > m) {
    string res = build(m, n, m - 1 - sx, sy, m - 1 - tx, ty);

    for (char &c : res) {
      if (c == 'D')
        c = 'L';
      else if (c == 'R')
        c = 'D';
      else if (c == 'U')
        c = 'R';
      else
        c = 'U';
    }

    return res;
  }

  // Reflect until sx <= tx.
  if (sx > tx) {
    string res = build(n, m, sy, m - 1 - sx, ty, m - 1 - tx);

    for (char &c : res) {
      if (c == 'L')
        c = 'R';
      else if (c == 'R')
        c = 'L';
    }

    return res;
  }

  // Reflect until sy <= ty.
  if (sy > ty) {
    string res = build(n, m, n - 1 - sy, sx, n - 1 - ty, tx);

    for (char &c : res) {
      if (c == 'U')
        c = 'D';
      else if (c == 'D')
        c = 'U';
    }

    return res;
  }

  if (n == 1)
    return string(m - 1, 'R');

  // Adjacent endpoints in a 2-row grid.
  if (n == 2 && sy == ty && sx + 1 == tx) {
    string res;

    res += string(sx, 'L');
    res += sy == 0 ? 'D' : 'U';
```

```
  res += string(m - 1, 'R');
  res += sy == 0 ? 'U' : 'D';
  res += string(m - 1 - tx, 'L');

  return res;
}

if (n <= 4 && m <= 5)
  return small_path(n, m, sy, sx, ty, tx);

// Strip two columns from the left.
if (sx >= 2 && possible(n, m - 2, sy, sx - 2, ty, tx - 2)) {
  string res = build(n, m - 2, sy, sx - 2, ty, tx - 2);

  int y = sy;
  int x = sx - 2;

  for (int i = 0; i < (int)res.size(); i++) {
    int d = id(res[i]);

    int ny = y + dy[d];
    int nx = x + dx[d];

    if (x == 0 && nx == 0) {
      return res.substr(0, i) + "L" + build(n, 2, y, 1, ny, 1) + "R" +
             res.substr(i + 1);
    }

    y = ny;
    x = nx;
  }
}

// Strip two columns from the right.
if (tx <= m - 3 && possible(n, m - 2, sy, sx, ty, tx)) {
  string res = build(n, m - 2, sy, sx, ty, tx);

  int y = sy;
  int x = sx;

  for (int i = 0; i < (int)res.size(); i++) {
    int d = id(res[i]);

    int ny = y + dy[d];
    int nx = x + dx[d];

    if (x == m - 3 && nx == m - 3) {
      return res.substr(0, i) + "R" + build(n, 2, y, 0, ny, 0) + "L" +
             res.substr(i + 1);
    }

    y = ny;
    x = nx;
  }
}

// Horizontal split.
for (int r = sy; r < ty; r++) {
  for (int c = 0; c < m; c++) {
    if (possible(r + 1, m, sy, sx, r, c) &&
        possible(n - r - 1, m, 0, c, ty - r - 1, tx)) {
      return build(r + 1, m, sy, sx, r, c) + "D" +
             build(n - r - 1, m, 0, c, ty - r - 1, tx);
    }
  }
}

// Vertical split.
for (int c = sx; c < tx; c++) {
  for (int r = 0; r < n; r++) {
    if (possible(n, c + 1, sy, sx, r, c) &&
        possible(n, m - c - 1, r, 0, ty, tx - c - 1)) {
      return build(n, c + 1, sy, sx, r, c) + "R" +
             build(n, m - c - 1, r, 0, ty, tx - c - 1);
    }
  }
}

  return "";
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int t;
  cin >> t;

  while (t--) {
    int n, m, y1, x1, y2, x2;
    cin >> n >> m >> y1 >> x1 >> y2 >> x2;

    --y1;
    --x1;
    --y2;
    --x2;

    if (!possible(n, m, y1, x1, y2, x2)) {
      cout << "NO\n";
      continue;
    }

    cout << "YES\n";
    cout << build(n, m, y1, x1, y2, x2) << '\n';
  }
}
```

Permutation Prime Sums

```
// cses/Construction Problems/permutation_prime_sums.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  int m = 2 * n + 5;

  vector<bool> prime(m, true);
  prime[0] = prime[1] = false;

  for (int i = 2; i * i < m; i++)
    if (prime[i])
      for (int j = i * i; j < m; j += i)
        prime[j] = false;

  // nextPrime[x] = smallest prime >= x
  vector<int> nextPrime(m);
  int nxt = -1;

  for (int i = m - 1; i >= 0; i--) {
    if (prime[i])
      nxt = i;
    nextPrime[i] = nxt;
  }

  vector<int> b(n + 1);

  int r = n;

  while (r > 0) {
    int p = nextPrime[r + 1];
    int l = p - r;

    for (int i = l; i <= r; i++)
      b[i] = p - i;

    r = l - 1;
  }

  for (int i = 1; i <= n; i++)
    cout << i << " \n"[i == n];

  for (int i = 1; i <= n; i++)
    cout << b[i] << " \n"[i == n];
}
```

Third Permutation

```
// cses/Construction Problems/third_permutation.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<int> a(n), b(n);
  for (int &x : a)
    cin >> x;
  for (int &x : b)
    cin >> x;

  if (n == 2) {
    cout << "IMPOSSIBLE\n";
    return 0;
  }

  // p[i] = position in a containing value b[i].
  vector<int> pos(n + 1);
  for (int i = 0; i < n; i++)
    pos[a[i]] = i;

  vector<int> p(n);
  for (int i = 0; i < n; i++)
    p[i] = pos[b[i]];

  // Concatenate the cycles of p.
  vector<int> order;
  vector<bool> vis(n);

  for (int i = 0; i < n; i++) {
    if (vis[i])
      continue;

    int u = i;
    while (!vis[u]) {
      vis[u] = true;
      order.push_back(u);
      u = p[u];
    }
  }

  // Shift by two positions in the concatenated cycle order.
  vector<int> c(n);

  for (int i = 0; i < n; i++)
    c[order[i]] = a[order[(i + 2) % n]];

  for (int x : c)
    cout << x << ' ';
  cout << '\n';
}
```

Advanced Graph Problems

Bus Companies

```cpp
// cses/Advanced Graph Problems/bus_companies.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<ll> cost(m);
  for (auto &x : cost)
    cin >> x;

  vector<vector<int>> companies(m);
  vector<vector<int>> at(n);

  for (int j = 0; j < m; j++) {
    int k;
    cin >> k;

    companies[j].resize(k);
    for (int &v : companies[j]) {
      cin >> v;
      --v;
      at[v].push_back(j);
    }
  }

  const ll INF = 4e18;
  vector<ll> distCity(n, INF), distCompany(m, INF);

  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
      pq;

  distCity[0] = 0;
  pq.push({0, 0});

  while (!pq.empty()) {
    auto [d, id] = pq.top();
    pq.pop();

    if (id < n) {
      int v = id;
      if (d != distCity[v])
        continue;

      for (int j : at[v]) {
        ll nd = d + cost[j];
        if (nd < distCompany[j]) {
          distCompany[j] = nd;
          pq.push({nd, n + j});
        }
      }
    } else {
      int j = id - n;
      if (d != distCompany[j])
        continue;

      for (int v : companies[j]) {
        if (d < distCity[v]) {
          distCity[v] = d;
          pq.push({d, v});
        }
      }
    }
  }

  for (int i = 0; i < n; i++)
    cout << distCity[i] << " \n"[i == n - 1];
}
```

Fixed Length Walk Queries

```cpp
// cses/Advanced Graph Problems/fixed_length_walk_queries.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Query {
  int b, id;
  ll x;
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m, q;
  cin >> n >> m >> q;

  vector<vector<int>> g(n);

  while (m--) {
    int a, b;
    cin >> a >> b;
    --a, --b;
    g[a].push_back(b);
    g[b].push_back(a);
  }

  vector<vector<Query>> queries(n);
  vector<char> ans(q);

  for (int i = 0; i < q; i++) {
    int a, b;
    ll x;
    cin >> a >> b >> x;
    --a, --b;
    queries[a].push_back({b, i, x});
  }

  for (int s = 0; s < n; s++) {
    if (queries[s].empty())
      continue;

    // state = 2 * node + parity
    vector<int> dist(2 * n, -1);
    queue<int> qu;

    dist[2 * s] = 0;
    qu.push(2 * s);

    while (!qu.empty()) {
      int state = qu.front();
      qu.pop();

      int u = state / 2;
      int p = state % 2;

      for (int v : g[u]) {
        int nxt = 2 * v + (p ^ 1);

        if (dist[nxt] == -1) {
          dist[nxt] = dist[state] + 1;
          qu.push(nxt);
        }
      }
    }

    for (auto [b, id, x] : queries[s]) {
      int d = dist[2 * b + (x & 1)];
      ans[id] = (d != -1 && d <= x);
    }
  }

  for (char ok : ans)
    cout << (ok ? "YES\n" : "NO\n");
}
```

Forbidden Cities

```cpp
// cses/Advanced Graph Problems/forbidden_cities.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int LOG = 20;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m, q;
  cin >> n >> m >> q;

  vector<vector<pair<int, int>>> g(n);

  for (int i = 0; i < m; i++) {
    int a, b;
    cin >> a >> b;
    --a, --b;
    g[a].push_back({b, i});
    g[b].push_back({a, i});
  }

  vector<int> tin(n, -1), tout(n), low(n), depth(n);
  vector<array<int, LOG>> up(n);

  int timer = 0;

  auto dfs = [&](auto &&self, int v, int p, int pe) -> void {
    tin[v] = low[v] = timer++;

    up[v][0] = p;
    for (int k = 1; k < LOG; k++)
      up[v][k] = up[up[v][k - 1]][k - 1];

    for (auto [to, id] : g[v]) {
      if (id == pe)
        continue;

      if (tin[to] != -1) {
        low[v] = min(low[v], tin[to]);
      } else {
        depth[to] = depth[v] + 1;
        self(self, to, v, id);
        low[v] = min(low[v], low[to]);
      }
    }

    tout[v] = timer - 1;
  };

  dfs(dfs, 0, 0, -1);

  auto ancestor = [&](int a, int b) {
    return tin[a] <= tin[b] && tin[b] <= tout[a];
  };

  auto jump = [&](int v, int d) {
    for (int k = 0; k < LOG; k++)
      if (d & (1 << k))
        v = up[v][k];
    return v;
  };

  // Component containing x after removing c.
  // -1 means the component on the parent side of c.
  auto side = [&](int c, int x) {
    if (!ancestor(c, x))
      return -1;

    int child = jump(x, depth[x] - depth[c] - 1);

    if (low[child] >= tin[c])
      return child;

    return -1;
  };

  while (q--) {
    int a, b, c;
    cin >> a >> b >> c;
    --a, --b, --c;
```

```
    if (a == c || b == c) {
      cout << "NO\n";
      continue;
    }

    cout << (side(c, a) == side(c, b) ? "YES\n" : "NO\n");
  }
}
```

Graph Coloring

```
// cses/Advanced Graph Problems/graph_coloring.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<int> adj(n);

  while (m--) {
    int a, b;
    cin >> a >> b;
    --a, --b;
    adj[a] |= 1 << b;
    adj[b] |= 1 << a;
  }

  int N = 1 << n;

  // independent[mask] = whether mask is an independent set
  vector<char> independent(N, false);
  independent[0] = true;

  for (int mask = 1; mask < N; ++mask) {
    int v = __builtin_ctz(mask);
    int rest = mask ^ (1 << v);

    independent[mask] = independent[rest] && !(adj[v] & rest);
  }

  // dp[mask] = minimum colors needed for vertices in mask
  vector<int> dp(N, n + 1), take(N);
  dp[0] = 0;

  for (int mask = 1; mask < N; ++mask) {
    int bit = mask & -mask;
    int rest = mask ^ bit;

    // The color class containing 'bit' can be any
    // independent subset containing it.
    for (int sub = rest;; sub = (sub - 1) & rest) {
      int group = sub | bit;

      if (independent[group] && dp[mask ^ group] + 1 < dp[mask]) {
        dp[mask] = dp[mask ^ group] + 1;
        take[mask] = group;
      }

      if (sub == 0)
        break;
    }
  }

  vector<int> color(n);
  int mask = N - 1;
  int c = 1;

  while (mask) {
    int group = take[mask];

    for (int v = 0; v < n; ++v)
      if (group & (1 << v))
        color[v] = c;

    mask ^= group;
    ++c;
  }

  cout << dp[N - 1] << '\n';

  for (int v = 0; v < n; ++v)
    cout << color[v] << " \n"[v == n - 1];
}
```

MST Edge Check

```
// cses/Advanced Graph Problems/mst_edge_check.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct DSU {
  vector<int> p, sz;

  DSU(int n) : p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }

  int find(int a) { return a == p[a] ? a : p[a] = find(p[a]); }

  void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b)
      return;
    if (sz[a] < sz[b])
      swap(a, b);
    p[b] = a;
    sz[a] += sz[b];
  }
};

struct Edge {
  int a, b, id;
  ll w;
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<Edge> e(m);
  for (int i = 0; i < m; i++) {
    cin >> e[i].a >> e[i].b >> e[i].w;
    e[i].id = i;
  }

  sort(e.begin(), e.end(),
       [](const Edge &x, const Edge &y) { return x.w < y.w; });

  DSU dsu(n);
  vector<bool> ans(m);

  for (int i = 0; i < m;) {
    int j = i;
    while (j < m && e[j].w == e[i].w)
      j++;

    // Check before joining edges of this weight.
    for (int k = i; k < j; k++)
      ans[e[k].id] = dsu.find(e[k].a) != dsu.find(e[k].b);

    for (int k = i; k < j; k++)
      dsu.unite(e[k].a, e[k].b);

    i = j;
  }

  for (bool x : ans)
    cout << (x ? "YES" : "NO") << '\n';
}
```

MST Edge Cost

```
// cses/Advanced Graph Problems/mst_edge_cost.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct DSU {
  vector<int> p, sz;

  DSU(int n) : p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }

  int find(int x) { return x == p[x] ? x : p[x] = find(p[x]); }

  bool unite(int a, int b) {
    a = find(a);
    b = find(b);

    if (a == b)
      return false;

    if (sz[a] < sz[b])
      swap(a, b);

    p[b] = a;
    sz[a] += sz[b];
    return true;
  }
};

struct Edge {
  int a, b, id;
  ll w;
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<Edge> edges(m);
  for (int i = 0; i < m; i++) {
    cin >> edges[i].a >> edges[i].b >> edges[i].w;
    edges[i].id = i;
  }

  vector<Edge> ord = edges;
  sort(ord.begin(), ord.end(),
       [](const Edge &a, const Edge &b) { return a.w < b.w; });

  DSU dsu(n);
  vector<vector<pair<int, ll>>> g(n + 1);
  ll mst = 0;

  for (auto [a, b, id, w] : ord) {
    if (dsu.unite(a, b)) {
      mst += w;
      g[a].push_back({b, w});
      g[b].push_back({a, w});
    }
  }

  int LOG = 1;
  while ((1 << LOG) <= n)
    LOG++;

  vector<int> depth(n + 1);
  vector<vector<int>> up(LOG, vector<int>(n + 1));
  vector<vector<ll>> mx(LOG, vector<ll>(n + 1));

  auto dfs = [&](auto &&self, int u, int p) -> void {
    for (auto [v, w] : g[u]) {
      if (v == p)
        continue;

      depth[v] = depth[u] + 1;
      up[0][v] = u;
      mx[0][v] = w;
      self(self, v, u);
    }
  };

  dfs(dfs, 1, 0);

  for (int j = 1; j < LOG; j++) {
    for (int v = 1; v <= n; v++) {
      up[j][v] = up[j - 1][up[j - 1][v]];
      mx[j][v] = max(mx[j - 1][v], mx[j - 1][up[j - 1][v]]);
    }
  }
```

```
  auto pathMax = [&](int a, int b) {
    ll ans = 0;

    if (depth[a] < depth[b])
      swap(a, b);

    int diff = depth[a] - depth[b];

    for (int j = LOG - 1; j >= 0; j--) {
      if (diff & (1 << j)) {
        ans = max(ans, mx[j][a]);
        a = up[j][a];
      }
    }

    if (a == b)
      return ans;

    for (int j = LOG - 1; j >= 0; j--) {
      if (up[j][a] != up[j][b]) {
        ans = max({ans, mx[j][a], mx[j][b]});
        a = up[j][a];
        b = up[j][b];
      }
    }

    ans = max({ans, mx[0][a], mx[0][b]});
    return ans;
  };

  for (auto [a, b, id, w] : edges)
    cout << mst + w - pathMax(a, b) << '\n';
}
```

MST Edge Set Check

```
// cses/Advanced Graph Problems/mst_edge_set_check.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct DSU {
  vector<int> p, sz;
  vector<pair<int, int>> hist;

  DSU(int n) : p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }

  int find(int x) {
    while (x != p[x])
      x = p[x];
    return x;
  }

  bool unite(int a, int b) {
    a = find(a);
    b = find(b);

    if (a == b)
      return false;

    if (sz[a] < sz[b])
      swap(a, b);

    hist.push_back({b, sz[a]});

    p[b] = a;
    sz[a] += sz[b];

    return true;
  }

  int snapshot() { return hist.size(); }

  void rollback(int snap) {
    while ((int)hist.size() > snap) {
      auto [b, oldSize] = hist.back();
      hist.pop_back();

      int a = p[b];

      p[b] = b;
      sz[a] = oldSize;
    }
  }
};

struct Edge {
  int u, v;
  ll w;
};

struct Event {
  ll w;
  int q, u, v;

  bool operator<(const Event &other) const {
    if (w != other.w)
      return w < other.w;
    return q < other.q;
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m, q;
  cin >> n >> m >> q;

  vector<Edge> edges(m);

  for (auto &[u, v, w] : edges)
    cin >> u >> v >> w;

  vector<Event> events;
  events.reserve(m);

  vector<char> ok(q, true);

  for (int qi = 0; qi < q; qi++) {
    int k;
    cin >> k;

    while (k--) {
      int id;
      cin >> id;
      --id;

      auto [u, v, w] = edges[id];
      events.push_back({w, qi, u, v});
    }
  }

  vector<int> ord(m);
  iota(ord.begin(), ord.end(), 0);

  sort(ord.begin(), ord.end(),
       [&](int a, int b) { return edges[a].w < edges[b].w; });

  sort(events.begin(), events.end());

  DSU dsu(n);
  int ptr = 0;

  for (int i = 0; i < (int)events.size();) {
    ll w = events[i].w;

    // Permanently add all strictly lighter edges.
    while (ptr < m && edges[ord[ptr]].w < w) {
      auto [u, v, _] = edges[ord[ptr++]];
      dsu.unite(u, v);
    }

    int j = i;

    // Test each query separately for this weight.
    while (j < (int)events.size() && events[j].w == w) {
      int qi = events[j].q;
      int snap = dsu.snapshot();

      int k = j;

      while (k < (int)events.size() && events[k].w == w && events[k].q == qi) {
        if (ok[qi] && !dsu.unite(events[k].u, events[k].v))
          ok[qi] = false;
        k++;
      }

      dsu.rollback(snap);
      j = k;
    }

    i = j;
  }

  for (bool x : ok)
    cout << (x ? "YES\n" : "NO\n");
}
```

Nearest Shops

```
// cses/Advanced Graph Problems/nearest_shops.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m, k;
  cin >> n >> m >> k;

  vector<int> shops(k);
  vector<bool> isShop(n + 1);

  for (int &x : shops) {
    cin >> x;
    isShop[x] = true;
  }

  vector<vector<int>> g(n + 1);
  vector<pair<int, int>> edges;

  for (int i = 0; i < m; i++) {
    int a, b;
    cin >> a >> b;
    g[a].push_back(b);
    g[b].push_back(a);
    edges.push_back({a, b});
  }

  // Multi-source BFS: owner[u] is the nearest shop to u.
  vector<int> dist(n + 1, -1), owner(n + 1, -1);
  queue<int> q;

  for (int s : shops) {
    dist[s] = 0;
    owner[s] = s;
    q.push(s);
  }

  while (!q.empty()) {
    int u = q.front();
    q.pop();

    for (int v : g[u]) {
      if (dist[v] == -1) {
        dist[v] = dist[u] + 1;
        owner[v] = owner[u];
        q.push(v);
      }
    }
  }

  // When two BFS regions touch, we get a path between their shops.
  const int INF = 1e9;
  vector<int> other(n + 1, INF);

  for (auto [u, v] : edges) {
    if (owner[u] != -1 && owner[v] != -1 && owner[u] != owner[v]) {
      int d = dist[u] + 1 + dist[v];
      other[owner[u]] = min(other[owner[u]], d);
      other[owner[v]] = min(other[owner[v]], d);
    }
  }
```

```
  for (int i = 1; i <= n; i++) {
    if (isShop[i]) {
      cout << (other[i] == INF ? -1 : other[i]);
    } else {
      cout << dist[i];
    }

    cout << (i == n ? '\n' : ' ');
  }
}
```

New Flight Routes

```
// cses/Advanced Graph Problems/new_flight_routes.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

vector<vector<int>> g, rg;
vector<int> order, comp;
vector<char> vis;

void dfs1(int u) {
  vis[u] = true;
  for (int v : g[u])
    if (!vis[v])
      dfs1(v);
  order.push_back(u);
}

void dfs2(int u, int c) {
  comp[u] = c;
  for (int v : rg[u])
    if (comp[v] == -1)
      dfs2(v, c);
}

struct Info {
  vector<int> src, sink;
  int comps;
};

Info getInfo(int n) {
  fill(vis.begin(), vis.end(), 0);
  fill(comp.begin(), comp.end(), -1);
  order.clear();

  for (int i = 0; i < n; ++i)
    if (!vis[i])
      dfs1(i);

  reverse(order.begin(), order.end());

  vector<int> rep;
  int cc = 0;

  for (int u : order) {
    if (comp[u] != -1)
      continue;

    rep.push_back(u);
    dfs2(u, cc++);
  }

  vector<char> in(cc), out(cc);

  for (int u = 0; u < n; ++u) {
    for (int v : g[u]) {
      if (comp[u] != comp[v]) {
        out[comp[u]] = true;
        in[comp[v]] = true;
      }
    }
  }

  Info res;
  res.comps = cc;

  for (int c = 0; c < cc; ++c) {
    if (!in[c])
      res.src.push_back(rep[c]);
    if (!out[c])
      res.sink.push_back(rep[c]);
  }

  return res;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  g.resize(n);
  rg.resize(n);
  vis.resize(n);
  comp.resize(n);
  order.reserve(n);

  while (m--) {
    int a, b;
    cin >> a >> b;
    --a, --b;

    g[a].push_back(b);
    rg[b].push_back(a);
  }

  mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

  vector<pair<int, int>> ans;
  Info cur = getInfo(n);

  while (cur.comps > 1 && cur.src.size() > 1 && cur.sink.size() > 1) {

    int k = min(cur.src.size(), cur.sink.size()) / 2;

    auto src = cur.src;
    auto sink = cur.sink;

    shuffle(src.begin(), src.end(), rng);
    shuffle(sink.begin(), sink.end(), rng);

    vector<pair<int, int>> add;

    for (int i = 0; i < k; ++i) {
      int u = sink[i];
      int v = src[i];

      g[u].push_back(v);
      rg[v].push_back(u);
      add.push_back({u, v});
    }

    Info nxt = getInfo(n);

    if ((int)nxt.src.size() != (int)cur.src.size() - k ||
        (int)nxt.sink.size() != (int)cur.sink.size() - k) {

      for (auto [u, v] : add) {
        g[u].pop_back();
        rg[v].pop_back();
      }
    } else {
      ans.insert(ans.end(), add.begin(), add.end());
      cur = std::move(nxt); // needs to use `std::` here.
    }
  }

  if (cur.comps > 1) {
    if (cur.src.size() == 1) {
      for (int t : cur.sink)
        ans.push_back({t, cur.src[0]});
    } else {
      for (int s : cur.src)
        ans.push_back({cur.sink[0], s});
    }
  }

  cout << ans.size() << '\n';
  for (auto [u, v] : ans)
    cout << u + 1 << ' ' << v + 1 << '\n';
}
```

Prüfer Code

```
// cses/Advanced Graph Problems/prufer_code.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<int> p(n - 2), deg(n + 1, 1);

  for (int &x : p) {
    cin >> x;
    deg[x]++;
  }

  priority_queue<int, vector<int>, greater<int>> leaves;

  for (int i = 1; i <= n; i++)
    if (deg[i] == 1)
      leaves.push(i);

  for (int x : p) {
    int u = leaves.top();
    leaves.pop();

    cout << u << ' ' << x << '\n';

    if (--deg[x] == 1)
      leaves.push(x);
  }

  int a = leaves.top();
  leaves.pop();
  int b = leaves.top();

  cout << a << ' ' << b << '\n';
}
```

Split into Two Paths

```
// cses/Advanced Graph Problems/split_into_two_paths.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<vector<int>> g(n), rg(n);
  vector<int> deg(n);

  while (m--) {
    int a, b;
    cin >> a >> b;
    --a, --b;
    g[a].push_back(b);
    rg[b].push_back(a);
    deg[b]++;
  }

  // Topological order.
  queue<int> q;
  for (int i = 0; i < n; ++i)
    if (!deg[i])
      q.push(i);

  vector<int> ord;
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    ord.push_back(u);

    for (int v : g[u])
      if (--deg[v] == 0)
        q.push(v);
  }
```

```
vector<int> pos(n);
for (int i = 0; i < n; ++i)
  pos[ord[i]] = i;

// At step i, ord[i] is the end of one path.
// active[y] means the other path may end at ord[y].
vector<int> active(n), prv(n, -2);

int version = 1;
bool empty = true;

for (int i = 0; i + 1 < n; ++i) {
  int v = ord[i + 1];

  bool straight = false;
  bool canSwitch = empty;
  int from = empty ? -1 : -2;

  for (int u : rg[v]) {
    int j = pos[u];

    if (j == i)
      straight = true;
    else if (j < i && active[j] == version && !canSwitch) {
      canSwitch = true;
      from = j;
    }
  }

  // Switch paths: ord[i+1] is appended to the other path,
  // so ord[i] becomes the new "other endpoint".
  if (canSwitch) {
    active[i] = version;
    prv[i] = from;
  }

  // Without ord[i] -> ord[i+1], all old states disappear.
  if (!straight) {
    if (!canSwitch) {
      cout << "NO\n";
      return 0;
    }

    ++version;
    empty = false;
    active[i] = version;
  }
}

int state = -1;

if (!empty) {
  state = -2;
  for (int i = n - 1; i >= 0; --i)
    if (active[i] == version) {
      state = i;
      break;
    }

  if (state == -2) {
    cout << "NO\n";
    return 0;
  }
}

// Reconstruct which path each topological position belongs to.
vector<int> side(n);
int cur = 0;

for (int i = n - 1; i > 0; --i) {
  side[i] = cur;

  if (state == i - 1) {
    state = prv[i - 1];
    cur ^= 1;
  }
}

side[0] = cur;

vector<int> path[2];

  for (int i = 0; i < n; ++i)
    path[side[i]].push_back(ord[i]);

  cout << "YES\n";

  for (int t = 0; t < 2; ++t) {
    cout << path[t].size();
    for (int u : path[t])
      cout << ' ' << u + 1;
    cout << '\n';
  }
}
```

Tree Coin Collecting I

```
// cses/Advanced Graph Problems/tree_coin_collecting_I.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

const int LOG = 20;
const int INF = 1e9;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, q;
  cin >> n >> q;

  vector<int> coin(n);
  for (int &x : coin)
    cin >> x;

  vector<vector<int>> adj(n);
  for (int i = 0; i < n - 1; i++) {
    int a, b;
    cin >> a >> b;
    --a, --b;
    adj[a].push_back(b);
    adj[b].push_back(a);
  }

  // Distance from every node to the nearest coin.
  vector<int> dist(n, INF);
  queue<int> qu;

  for (int i = 0; i < n; i++) {
    if (coin[i]) {
      dist[i] = 0;
      qu.push(i);
    }
  }

  while (!qu.empty()) {
    int v = qu.front();
    qu.pop();

    for (int u : adj[v]) {
      if (dist[u] > dist[v] + 1) {
        dist[u] = dist[v] + 1;
        qu.push(u);
      }
    }
  }

  vector<int> depth(n);
  vector<array<int, LOG>> up(n);
  vector<array<int, LOG>> mn(n);

  // Root the tree at 0.
  vector<int> par(n, -1);
  par[0] = 0;
  qu.push(0);

  while (!qu.empty()) {
    int v = qu.front();
    qu.pop();

    for (int u : adj[v]) {
      if (u == par[v])
        continue;
      par[u] = v;
      depth[u] = depth[v] + 1;
      qu.push(u);
    }
  }

  for (int v = 0; v < n; v++) {
    up[v][0] = par[v];
    mn[v][0] = min(dist[v], dist[par[v]]);
  }

  for (int j = 1; j < LOG; j++) {
    for (int v = 0; v < n; v++) {
      up[v][j] = up[up[v][j - 1]][j - 1];
      mn[v][j] = min(mn[v][j - 1], mn[up[v][j - 1]][j - 1]);
    }
  }

  auto lca = [&](int a, int b) {
    if (depth[a] < depth[b])
      swap(a, b);

    int d = depth[a] - depth[b];
    for (int j = 0; j < LOG; j++)
      if (d >> j & 1)
        a = up[a][j];

    if (a == b)
      return a;

    for (int j = LOG - 1; j >= 0; j--) {
      if (up[a][j] != up[b][j]) {
        a = up[a][j];
        b = up[b][j];
      }
    }

    return up[a][0];
  };

  auto pathMinToAncestor = [&](int v, int anc) {
    int ans = dist[v];
    int d = depth[v] - depth[anc];

    for (int j = 0; j < LOG; j++) {
      if (d >> j & 1) {
        ans = min(ans, mn[v][j]);
        v = up[v][j];
      }
    }

    return min(ans, dist[anc]);
  };

  while (q--) {
    int a, b;
    cin >> a >> b;
    --a, --b;

    int c = lca(a, b);

    int pathDist = depth[a] + depth[b] - 2 * depth[c];
    int best = min(pathMinToAncestor(a, c), pathMinToAncestor(b, c));

    cout << pathDist + 2 * best << '\n';
  }
}
```

Tree Coin Collecting II

```
// cses/Advanced Graph Problems/tree_coin_collecting_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, q;
  cin >> n >> q;

  vector<int> coin(n);
```

```
int coins = 0;

for (int &x : coin) {
  cin >> x;
  coins += x;
}

vector<vector<int>> g(n);

for (int i = 0; i < n - 1; i++) {
  int a, b;
  cin >> a >> b;
  --a, --b;

  g[a].push_back(b);
  g[b].push_back(a);
}

const int LOG = 20;

vector<int> parent(n, -1), depth(n), order;
vector<array<int, LOG>> up(n);

parent[0] = 0;

vector<int> st = {0};

while (!st.empty()) {
  int u = st.back();
  st.pop_back();

  order.push_back(u);

  for (int v : g[u]) {
    if (v == parent[u])
      continue;

    parent[v] = u;
    depth[v] = depth[u] + 1;
    st.push_back(v);
  }
}

for (int v = 0; v < n; v++)
  up[v][0] = parent[v];

for (int j = 1; j < LOG; j++) {
  for (int v = 0; v < n; v++) {
    up[v][j] = up[up[v][j - 1]][j - 1];
  }
}
```

```
auto lca = [&](int a, int b) {
  if (depth[a] < depth[b])
    swap(a, b);

  int d = depth[a] - depth[b];

  for (int j = 0; j < LOG; j++) {
    if (d >> j & 1)
      a = up[a][j];
  }

  if (a == b)
    return a;

  for (int j = LOG - 1; j >= 0; j--) {
    if (up[a][j] != up[b][j]) {
      a = up[a][j];
      b = up[b][j];
    }
  }

  return parent[a];
};

auto dist = [&](int a, int b) {
  int c = lca(a, b);
  return depth[a] + depth[b] - 2 * depth[c];
};

// Find the minimal subtree containing all coins.
vector<int> sub = coin;

for (int i = n - 1; i > 0; i--) {
  int v = order[i];
  sub[parent[v]] += sub[v];
}

vector<char> core(n, false);
ll coreEdges = 0;

for (int v = 0; v < n; v++) {
  if (coin[v])
    core[v] = true;
}

for (int v = 1; v < n; v++) {
  // This edge separates coins on both sides,
  // so it belongs to the coin subtree.
  if (sub[v] > 0 && sub[v] < coins) {
    core[v] = true;
    core[parent[v]] = true;
    coreEdges++;
  }
}
```

```
  }

  // For every node:
  // root[v]   = which core node its branch attaches to
  // toCore[v] = distance to the core
  vector<int> root(n, -1), toCore(n, -1);
  queue<int> qu;

  for (int v = 0; v < n; v++) {
    if (core[v]) {
      root[v] = v;
      toCore[v] = 0;
      qu.push(v);
    }
  }

  while (!qu.empty()) {
    int u = qu.front();
    qu.pop();

    for (int v : g[u]) {
      if (toCore[v] != -1)
        continue;

      root[v] = root[u];
      toCore[v] = toCore[u] + 1;
      qu.push(v);
    }
  }

  while (q--) {
    int a, b;
    cin >> a >> b;
    --a, --b;

    ll ans;

    if (root[a] == root[b]) {
      // Their branches meet before entering the coin subtree.
      ans = 2 * coreEdges + toCore[a] + toCore[b];
    } else {
      // Both branches are added independently.
      ll requiredEdges = coreEdges + toCore[a] + toCore[b];

      ans = 2 * requiredEdges - dist(a, b);
    }

    cout << ans << '\n';
  }
}
```

Counting Problems

All Letter Subgrid Count I

```cpp
// cses/Counting Problems/all_letter_subgrid_count_I.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

static unsigned short dp[2][3001][26];

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  const int INF = 6000;

  for (int t = 0; t < 2; t++)
    for (int j = 0; j <= n; j++)
      for (int c = 0; c < k; c++)
        dp[t][j][c] = INF;

  ll ans = 0;

  for (int i = 1; i <= n; i++) {
    string s;
    cin >> s;

    int cur = i & 1;
    int pre = cur ^ 1;

    for (int c = 0; c < k; c++)
      dp[cur][0][c] = INF;

    for (int j = 1; j <= n; j++) {
      int here = s[j - 1] - 'A';
      int need = 1;

      for (int c = 0; c < here; c++) {
        int x = min({dp[pre][j][c], dp[cur][j - 1][c], dp[pre][j - 1][c]}) + 1;

        dp[cur][j][c] = x;
        need = max(need, x);
      }

      dp[cur][j][here] = 1;

      for (int c = here + 1; c < k; c++) {
        int x = min({dp[pre][j][c], dp[cur][j - 1][c], dp[pre][j - 1][c]}) + 1;

        dp[cur][j][c] = x;
        need = max(need, x);
      }

      ans += max(0, min(i, j) - need + 1);
    }
  }

  cout << ans << '\n';
}
```

All Letter Subgrid Count II

```cpp
// cses/Counting Problems/all_letter_subgrid_count_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 500;

struct OrQueue {
  int val[N], in_or[N], out_or[N];
  int in = 0, out = 0;

  void clear() { in = out = 0; }

  void push(int x) {
    val[in] = x;
    in_or[in] = x | (in ? in_or[in - 1] : 0);
    ++in;
  }

  void pop() {
    if (!out) {
      while (in) {
        int x = val[--in];
        out_or[out] = x | (out ? out_or[out - 1] : 0);
        ++out;
      }
    }
    --out;
  }

  int get() const {
    return (in ? in_or[in - 1] : 0) | (out ? out_or[out - 1] : 0);
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<string> g(n);
  for (auto &s : g)
    cin >> s;

  int full = (1 << k) - 1;
  int col[N];
  OrQueue q;

  ll ans = 0;

  for (int top = 0; top < n; ++top) {
    fill(col, col + n, 0);

    for (int bottom = top; bottom < n; ++bottom) {
      int all = 0;

      for (int c = 0; c < n; ++c) {
        col[c] |= 1 << (g[bottom][c] - 'A');
        all |= col[c];
      }

      if (all != full)
        continue;

      q.clear();

      for (int right = 0; right < n; ++right) {
        q.push(col[right]);

        while (q.get() == full) {
          ans += n - right;
          q.pop();
        }
      }
    }
  }

  cout << ans << '\n';
}
```

Border Subgrid Count I

```cpp
// cses/Counting Problems/border_subgrid_count_I.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct FastPrefix {
  int blocks;
  vector<int> bit;
  vector<unsigned long long> mask;

  FastPrefix(int n) : blocks((n + 63) / 64), bit(blocks + 1), mask(blocks) {}

  void clear() {
    fill(bit.begin(), bit.end(), 0);
    fill(mask.begin(), mask.end(), 0);
  }

  void add(int x) {
    int b = x / 64;
    mask[b] |= 1ULL << (x % 64);

    for (int i = b + 1; i <= blocks; i += i & -i)
      ++bit[i];
  }

  // number of added positions in [0, x)
  int sum(int x) {
    int b = x / 64, res = 0;

    for (int i = b; i > 0; i -= i & -i)
      res += bit[i];

    if (b < blocks && x % 64)
      res += __builtin_popcountll(mask[b] & ((1ULL << (x % 64)) - 1));

    return res;
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<string> g(n);
  for (auto &row : g)
    cin >> row;

  int sz = n * n;

  // reach1: min(same-letter run right, down)
  // reach2: min(same-letter run left, up)
  vector<unsigned short> reach1(sz), reach2(sz);
  vector<int> vert(n);

  for (int i = n - 1; i >= 0; --i) {
    int right = 0;

    for (int j = n - 1; j >= 0; --j) {
      right = (j + 1 < n && g[i][j] == g[i][j + 1]) ? right + 1 : 1;

      vert[j] = (i + 1 < n && g[i][j] == g[i + 1][j]) ? vert[j] + 1 : 1;

      reach1[i * n + j] = min(right, vert[j]);
    }
  }

  fill(vert.begin(), vert.end(), 0);

  for (int i = 0; i < n; ++i) {
    int left = 0;

    for (int j = 0; j < n; ++j) {
      left = (j > 0 && g[i][j] == g[i][j - 1]) ? left + 1 : 1;

      vert[j] = (i > 0 && g[i][j] == g[i - 1][j]) ? vert[j] + 1 : 1;

      reach2[i * n + j] = min(left, vert[j]);
    }
  }

  vector<ll> ans(k);

  vector<unsigned short> start_reach(n), end_reach(n);
  vector<unsigned char> color(n);

  vector<int> head(n + 1), nxt(n);
  FastPrefix dead_pos(n);
```

```
  for (int d = -n + 1; d < n; ++d) {
    int i0 = max(0, -d);
    int j0 = max(0, d);
    int len = n - abs(d);

    // Copy the diagonal contiguously for better cache behavior.
    for (int t = 0; t < len; ++t) {
      int i = i0 + t;
      int j = j0 + t;
      int p = i * n + j;

      start_reach[t] = reach1[p];
      end_reach[t] = reach2[p];
      color[t] = g[i][j] - 'A';
    }

    fill(head.begin(), head.begin() + len + 1, -1);
    dead_pos.clear();

    int dead = 0;

    for (int t = 0; t < len; ++t) {
      // Top-left corners that can no longer reach t.
      for (int p = head[t]; p != -1; p = nxt[p]) {
        dead_pos.add(p);
        ++dead;
      }

      // Corner t remains usable through t + start_reach[t] - 1.
      int expire = t + start_reach[t];
      nxt[t] = head[expire];
      head[expire] = t;

      // Bottom-right corner t can pair only with
      // positions [left, t].
      int left = t + 1 - end_reach[t];

      int dead_here = dead - dead_pos.sum(left);
      ans[color[t]] += end_reach[t] - dead_here;
    }
  }

  for (ll x : ans)
    cout << x << '\n';
}
```

Border Subgrid Count II

```
// cses/Counting Problems/border_subgrid_count_II.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<string> g(n);
  for (auto &s : g)
    cin >> s;

  // down[i][j] = number of equal letters starting at (i,j) going down
  vector<vector<int>> down(n, vector<int>(n, 1));

  for (int i = n - 2; i >= 0; --i)
    for (int j = 0; j < n; ++j)
      if (g[i][j] == g[i + 1][j])
        down[i][j] = down[i + 1][j] + 1;

  vector<ll> ans(k);

  for (int top = 0; top < n; ++top) {
    for (int bot = top; bot < n; ++bot) {
      int h = bot - top + 1;
      char cur = 0;
      ll cnt = 0;

      for (int col = 0; col < n; ++col) {
        if (g[top][col] != g[bot][col]) {
          cur = 0;
          cnt = 0;
          continue;
        }

        char c = g[top][col];

        // Start of a new horizontal run.
        if (c != cur) {
          cur = c;
          cnt = 0;
        }

        // This column can be the right border.
        if (down[top][col] >= h) {
          ++cnt;
          ans[c - 'A'] += cnt;
        }
      }
    }
  }

  for (ll x : ans)
    cout << x << '\n';
}
```

Collecting Numbers Distribution

```
// cses/Counting Problems/collecting_numbers_distribution.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<ll> dp(n + 1);
  dp[1] = 1;

  for (int i = 2; i <= n; i++) {
    for (int k = i; k >= 1; k--) {
      dp[k] = (k * dp[k] + (i - k + 1LL) * dp[k - 1]) % MOD;
    }
  }

  for (int k = 1; k <= n; k++)
    cout << dp[k] << '\n';
}
```

Counting Reorders

```
// cses/Counting Problems/counting_reorders.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 1e9 + 7;

ll modpow(ll a, ll b) {
  ll r = 1;
  while (b) {
    if (b & 1)
      r = r * a % MOD;
    a = a * a % MOD;
    b >>= 1;
  }
  return r;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  string s;
  cin >> s;
  int n = s.size();

  vector<int> cnt(26);
  for (char c : s)
    cnt[c - 'a']++;

  vector<ll> fact(n + 1), ifact(n + 1);
  fact[0] = 1;
  for (int i = 1; i <= n; i++)
    fact[i] = fact[i - 1] * i % MOD;

  ifact[n] = modpow(fact[n], MOD - 2);
  for (int i = n; i; i--)
    ifact[i - 1] = ifact[i] * i % MOD;

  // dp[j] = coefficient for selecting j forced equal-adjacency bonds
  vector<ll> dp(n + 1);
  dp[0] = 1;

  int deg = 0;

  for (int c : cnt) {
    if (!c)
      continue;

    vector<ll> ndp(n + 1);

    for (int j = 0; j < c; j++) {
      // C(c-1, j) / (c-j)!
      ll ways = fact[c - 1] * ifact[j] % MOD * ifact[c - 1 - j] % MOD *
                ifact[c - j] % MOD;

      for (int k = 0; k <= deg; k++) {
        ndp[k + j] = (ndp[k + j] + dp[k] * ways) % MOD;
      }
    }

    deg += c - 1;
    dp.swap(ndp);
  }

  ll ans = 0;

  for (int j = 0; j <= deg; j++) {
    ll cur = dp[j] * fact[n - j] % MOD;

    if (j & 1)
      ans = (ans - cur + MOD) % MOD;
    else
      ans = (ans + cur) % MOD;
  }

  cout << ans << '\n';
}
```

Filled Subgrid Count I

```
// cses/Counting Problems/filled_subgrid_count_I.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<ll> ans(k);
  vector<int> prev(n), cur(n);

  string last, row;

  for (int i = 0; i < n; i++) {
    cin >> row;

    for (int j = 0; j < n; j++) {
      cur[j] = 1;

      if (i > 0 && j > 0 && row[j] == last[j] && row[j] == row[j - 1] &&
          row[j] == last[j - 1]) {
        cur[j] = 1 + min({prev[j], cur[j - 1], prev[j - 1]});
      }

      ans[row[j] - 'A'] += cur[j];
    }
```

```
    swap(prev, cur);
    last = row;
  }

  for (ll x : ans)
    cout << x << '\n';
}
```

Filled Subgrid Count II

```
// cses/Counting Problems/filled_subgrid_count_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<ll> ans(k);
  vector<int> h(n);
  string prev(n, '?');

  for (int i = 0; i < n; i++) {
    string s;
    cin >> s;

    for (int j = 0; j < n; j++) {
      if (i && s[j] == prev[j])
        h[j]++;
      else
        h[j] = 1;
    }

    vector<pair<int, int>> st;
    ll sum = 0;

    for (int j = 0; j < n; j++) {
      if (j && s[j] != s[j - 1]) {
        st.clear();
        sum = 0;
      }

      int cnt = 1;

      while (!st.empty() && st.back().first >= h[j]) {
        auto [x, c] = st.back();
        st.pop_back();

        sum -= 1LL * x * c;
        cnt += c;
      }

      st.push_back({h[j], cnt});
      sum += 1LL * h[j] * cnt;

      ans[s[j] - 'A'] += sum;
    }

    prev = s;
  }

  for (ll x : ans)
    cout << x << '\n';
}
```

Raab Game II

```
// cses/Counting Problems/raab_game_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 1e9 + 7;

ll pw(ll a, ll b) {
  ll r = 1;
  while (b) {
    if (b & 1)
      r = r * a % MOD;
    a = a * a % MOD;
    b >>= 1;
  }
  return r;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int t;
  cin >> t;

  vector<array<int, 3>> q(t);
  int N = 0;

  for (auto &[n, a, b] : q) {
    cin >> n >> a >> b;
    N = max(N, n);
  }

  vector<ll> fact(N + 1, 1), invfact(N + 1, 1);
  for (int i = 1; i <= N; i++)
    fact[i] = fact[i - 1] * i % MOD;

  invfact[N] = pw(fact[N], MOD - 2);
  for (int i = N; i; i--)
    invfact[i - 1] = invfact[i] * i % MOD;

  // Need dp[a][b] only for queried pairs.
  vector<vector<int>> ask(N + 1);
  vector<ll> ways(t);

  for (int i = 0; i < t; i++) {
    auto [n, a, b] = q[i];
    if (a + b <= n)
      ask[a].push_back(i);
  }

  // dp[a][b]:
  // derangements of a+b elements with
  // a positions p[i] < i and b positions p[i] > i.
  vector<ll> prv(N + 1), cur(N + 1);

  for (int a = 0; a <= N; a++) {
    fill(cur.begin(), cur.end(), 0);

    if (a == 0)
      cur[0] = 1;

    for (int b = 0; a + b <= N; b++) {
      if (a == 0 && b == 0)
        continue;

      ll x = 0;

      if (b)
        x += (ll)a * cur[b - 1];

      if (a)
        x += (ll)b * prv[b];

      if (a && b)
        x += (ll)(a + b - 1) * prv[b - 1];

      cur[b] = x % MOD;
    }

    for (int id : ask[a])
      ways[id] = cur[q[id][2]];

    swap(prv, cur);
  }

  for (int i = 0; i < t; i++) {
    auto [n, a, b] = q[i];

    if (a + b > n) {
      cout << 0 << '\n';
      continue;
    }

    int m = a + b;

    ll choose = fact[n] * invfact[m] % MOD * invfact[n - m] % MOD;

    ll ans = fact[n] * choose % MOD * ways[i] % MOD;
    cout << ans << '\n';
  }
}
```

Tournament Graph Distribution

```
// cses/Counting Problems/tournament_graph_distribution.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 1'000'000'007;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  static ll C[501][501];
  static ll dp[501][501];

  vector<ll> all(n + 1), strong(n + 1);

  // Binomial coefficients
  for (int i = 0; i <= n; i++) {
    C[i][0] = C[i][i] = 1;
    for (int j = 1; j < i; j++)
      C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % MOD;
  }

  // all[i] = number of tournaments on i vertices
  //        = 2^(i choose 2)
  all[0] = 1;
  ll p2 = 1;
  for (int i = 1; i <= n; i++) {
    all[i] = all[i - 1] * p2 % MOD;
    p2 = p2 * 2 % MOD;
  }

  // strong[i] = number of strongly connected tournaments on i vertices.
  // Pick the first SCC; all its edges point towards the remaining vertices.
  for (int i = 1; i <= n; i++) {
    strong[i] = all[i];

    for (int s = 1; s < i; s++) {
      ll ways = C[i][s] * strong[s] % MOD * all[i - s] % MOD;
      strong[i] -= ways;
      if (strong[i] < 0)
        strong[i] += MOD;
    }
  }

  // dp[i][k] = tournaments on i vertices with exactly k SCCs.
  dp[0][0] = 1;

  for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= i; s++) {
      ll ways = C[i][s] * strong[s] % MOD;

      for (int k = 1; k <= i - s + 1; k++) {
        dp[i][k] += ways * dp[i - s][k - 1] % MOD;
        if (dp[i][k] >= MOD)
          dp[i][k] -= MOD;
      }
    }
  }

  for (int k = 1; k <= n; k++)
    cout << dp[n][k] << '\n';
}
```

Additional Problems I

Beautiful Permutation II

```cpp
// cses/Additional Problems I/beautiful_permutation_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  if (n == 2 || n == 3) {
    cout << "NO SOLUTION\n";
    return 0;
  }

  set<int> rem;
  for (int i = 1; i <= n; i++)
    rem.insert(i);

  vector<int> ans;

  // Greedily construct the prefix.
  // Leave 5 elements so completion is always possible.
  while ((int)rem.size() > 5) {
    auto it = rem.begin();

    while (!ans.empty() && abs(*it - ans.back()) == 1)
      ++it;

    ans.push_back(*it);
    rem.erase(it);
  }

  // Brute force the last at most 5 elements.
  vector<int> tail(rem.begin(), rem.end());

  do {
    bool ok = ans.empty() || abs(ans.back() - tail[0]) != 1;

    for (int i = 1; i < (int)tail.size(); i++)
      if (abs(tail[i] - tail[i - 1]) == 1)
        ok = false;

    if (ok) {
      ans.insert(ans.end(), tail.begin(), tail.end());
      break;
    }
  } while (next_permutation(tail.begin(), tail.end()));

  for (int i = 0; i < n; i++)
    cout << ans[i] << " \n"[i + 1 == n];
}
```

Bubble Sort Rounds II

```cpp
// cses/Additional Problems I/bubble_sort_rounds_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  ll k;
  cin >> n >> k;

  vector<int> a(n);
  for (int &x : a)
    cin >> x;

  int r = min<ll>(n - 1, k);

  priority_queue<pair<int, int>, vector<pair<int, int>>,
                 greater<pair<int, int>>>
      pq;

  for (int i = 0; i <= r; i++)
    pq.push({a[i], i});

  int nxt = r + 1;

  for (int i = 0; i < n; i++) {
    auto [x, id] = pq.top();
    pq.pop();

    cout << x << " \n"[i == n - 1];

    if (nxt < n) {
      pq.push({a[nxt], nxt});
      nxt++;
    }
  }
}
```

Distinct Values Splits

```cpp
// cses/Additional Problems I/distinct_values_splits.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<int> a(n);
  for (int &x : a)
    cin >> x;

  vector<ll> dp(n + 1), sum(n + 1);
  unordered_map<int, int> last;
  last.reserve(2 * n);

  dp[0] = sum[0] = 1;

  int l = 0;

  for (int i = 0; i < n; i++) {
    if (last.count(a[i]))
      l = max(l, last[a[i]] + 1);

    last[a[i]] = i;

    dp[i + 1] = sum[i];
    if (l > 0)
      dp[i + 1] = (dp[i + 1] - sum[l - 1] + MOD) % MOD;

    sum[i + 1] = (sum[i] + dp[i + 1]) % MOD;
  }

  cout << dp[n] << '\n';
}
```

Distinct Values Sum

```cpp
// cses/Additional Problems I/distinct_values_sum.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  unordered_map<int, int> last;
  last.reserve(2 * n);

  ll cur = 0, ans = 0;

  for (int i = 1; i <= n; i++) {
    int x;
    cin >> x;

    cur += i - last[x];
    last[x] = i;

    ans += cur;
  }

  cout << ans << '\n';
}
```

Letter Pair Move Game

```cpp
// cses/Additional Problems I/letter_pair_move_game.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

map<string, string> build_bfs(const vector<string> &roots) {
  map<string, string> par;
  queue<string> q;

  for (auto s : roots) {
    if (!par.count(s)) {
      par[s] = "";
      q.push(s);
    }
  }

  int m = roots[0].size();

  while (!q.empty()) {
    string s = q.front();
    q.pop();

    int h = s.find("..");

    for (int i = 0; i + 1 < m; ++i) {
      if (s[i] == '.' || s[i + 1] == '.')
        continue;

      string t = s;
      swap(t[i], t[h]);
      swap(t[i + 1], t[h + 1]);

      if (!par.count(t)) {
        par[t] = s;
        q.push(t);
      }
    }
  }

  return par;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  string s;
  cin >> n >> s;

  vector<string> ans;

  auto move_pair = [&](int i, int j) {
    swap(s[i], s[j]);
    swap(s[i + 1], s[j + 1]);
    ans.push_back(s);
  };

  // Small cases: exact BFS.
  if (n <= 3) {
    string base(n - 1, 'A');
    base += string(n - 1, 'B');

    vector<string> roots;
```

```
  for (int h = 0; h < 2 * n - 1; ++h) {
    string t = base;
    t.insert(h, "..");
    roots.push_back(t);
  }

  auto par = build_bfs(roots);

  if (!par.count(s)) {
    cout << -1 << '\n';
    return 0;
  }

  while (!par[s].empty()) {
    s = par[s];
    ans.push_back(s);
  }
} else {
  int m = 2 * n;

  // Move the empty pair to positions 0,1.
  int h = s.find("..");

  if (h > 1) {
    move_pair(h, 0);
  } else if (h == 1) {
    move_pair(1, 3);
    move_pair(3, 0);
  }

  // All 448 possible 8-cell states are reachable from one
  // of these sorted states.
  vector<string> roots;

  for (int a = 0; a <= 6; ++a)
    roots.push_back(".." + string(a, 'A') + string(6 - a, 'B'));

  auto par = build_bfs(roots);

  auto sort8 = [&](int l) {
    string cur = s.substr(l, 8);

    while (!par[cur].empty()) {
      cur = par[cur];
      s.replace(l, 8, cur);
      ans.push_back(s);
    }
  };

  int p = 0; // current position of ".."

  while (p + 8 < m) {
    // Fix two A's permanently.
    if (s[p + 2] == 'A' && s[p + 3] == 'A') {
      move_pair(p, p + 2);
      p += 2;
      continue;
    }

    // There is a B among the first two letters after "..".
    int b = (int)s.find('B', p + 2);
    size_t pos = s.find('A', b + 2);

    // The remaining disorder is inside the next 8 cells.
    if (pos == string::npos) {
      sort8(p);
      break;
    }

    int a = (int)pos;
    bool last = (a == m - 1);

    // If the last A is at the final position, use the BA pair.
    if (last)
      --a;

    // Swap the pair starting at b with the pair starting at a,
    // using ".." as temporary storage.
    move_pair(p, b);
    move_pair(b, a);
    move_pair(a, p);
    if (last)
      sort8(p);
  }

  sort8(p);
}

cout << ans.size() << '\n';
for (auto &t : ans)
  cout << t << '\n';
}
```

Maximum Average Subarrays

```
// cses/Additional Problems I/maximum_average_subarrays.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<ll> s(n + 1);
  for (int i = 1; i <= n; i++) {
    ll x;
    cin >> x;
    s[i] = s[i - 1] + x;
  }

  auto cross = [&](int a, int b, int c) {
    return 1LL * (b - a) * (s[c] - s[a]) - 1LL * (c - a) * (s[b] - s[a]);
  };

  vector<int> hull;
  hull.push_back(0);

  for (int i = 1; i <= n; i++) {
    int l = 0, r = (int)hull.size() - 1;

    while (l < r) {
      int m = (l + r) / 2;

      // slope(hull[m], i) >= slope(hull[m+1], i)
      if (cross(hull[m], hull[m + 1], i) <= 0)
        r = m;
      else
        l = m + 1;
    }

    cout << i - hull[l] << " \n"[i == n];

    while (hull.size() >= 2 &&
           cross(hull[hull.size() - 2], hull.back(), i) <= 0) {
      hull.pop_back();
    }

    hull.push_back(i);
  }
}
```

Nearest Campsites I

```
// cses/Additional Problems I/nearest_campsites_I.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct BIT {
  vector<int> t;

  BIT(int n) : t(n + 1, INT_MIN) {}

  void add(int i, int v) {
    for (; i < (int)t.size(); i += i & -i)
      t[i] = max(t[i], v);
  }

  int get(int i) {
    int res = INT_MIN;
    for (; i; i -= i & -i)
      res = max(res, t[i]);
    return res;
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<pair<int, int>> reserved(n), freec(m);

  for (auto &[x, y] : reserved)
    cin >> x >> y;
  for (auto &[x, y] : freec)
    cin >> x >> y;

  const int INF = 1e9;
  vector<int> best(m, INF);

  for (int sx : {-1, 1}) {
    for (int sy : {-1, 1}) {
      vector<pair<int, int>> p(n);
      vector<array<int, 3>> q(m);
      vector<int> ys;

      for (int i = 0; i < n; ++i) {
        p[i] = {sx * reserved[i].first, sy * reserved[i].second};
        ys.push_back(p[i].second);
      }

      for (int i = 0; i < m; ++i) {
        q[i] = {sx * freec[i].first, sy * freec[i].second, i};
      }

      sort(p.begin(), p.end());
      sort(q.begin(), q.end());

      sort(ys.begin(), ys.end());
      ys.erase(unique(ys.begin(), ys.end()), ys.end());

      BIT bit(ys.size());
      int j = 0;

      for (auto [x, y, id] : q) {
        while (j < n && p[j].first <= x) {
          int pos =
              lower_bound(ys.begin(), ys.end(), p[j].second) - ys.begin() + 1;

          bit.add(pos, p[j].first + p[j].second);
          ++j;
        }

        int pos = upper_bound(ys.begin(), ys.end(), y) - ys.begin();

        int v = bit.get(pos);

        if (v != INT_MIN)
          best[id] = min(best[id], x + y - v);
      }
    }
  }

  cout << *max_element(best.begin(), best.end()) << '\n';
}
```

Nearest Campsites II

```
// cses/Additional Problems I/nearest_campsites_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Fenwick {
  int n;
  vector<int> bit;

  Fenwick(int n) : n(n), bit(n + 1, INT_MIN) {}

  void update(int i, int x) {
    for (; i <= n; i += i & -i)
      bit[i] = max(bit[i], x);
  }
```

```
  int query(int i) {
    int res = INT_MIN;
    for (; i; i -= i & -i)
      res = max(res, bit[i]);
    return res;
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<pair<int, int>> a(n), q(m);
  for (auto &[x, y] : a)
    cin >> x >> y;
  for (auto &[x, y] : q)
    cin >> x >> y;

  vector<int> ans(m, INT_MAX);

  for (int sx : {-1, 1}) {
    for (int sy : {-1, 1}) {
      // {x, type, y, id}; reserved (type 0) before queries on ties
      vector<array<int, 4>> ev;
      vector<int> ys;
      ev.reserve(n + m);
      ys.reserve(n + m);

      for (auto [x, y] : a) {
        x *= sx;
        y *= sy;
        ev.push_back({x, 0, y, -1});
        ys.push_back(y);
      }

      for (int i = 0; i < m; ++i) {
        auto [x, y] = q[i];
        x *= sx;
        y *= sy;
        ev.push_back({x, 1, y, i});
        ys.push_back(y);
      }

      sort(ev.begin(), ev.end());
      sort(ys.begin(), ys.end());
      ys.erase(unique(ys.begin(), ys.end()), ys.end());

      Fenwick fw(ys.size());

      for (auto [x, type, y, id] : ev) {
        int p = lower_bound(ys.begin(), ys.end(), y) - ys.begin() + 1;

        if (type == 0) {
          fw.update(p, x + y);
        } else {
          int best = fw.query(p);
          if (best != INT_MIN)
            ans[id] = min(ans[id], x + y - best);
        }
      }
    }
  }

  for (int x : ans)
    cout << x << ' ';
  cout << '\n';
}
```

Permutation Subsequence

```
// cses/Additional Problems I/permutation_subsequence.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<int> pos(n + 1);
  for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    pos[x] = i;
  }

  vector<int> val, p;
  for (int i = 0; i < m; i++) {
    int x;
    cin >> x;
    if (x <= n) {
      val.push_back(x);
      p.push_back(pos[x]);
    }
  }

  int k = p.size();
  vector<int> tail, tail_idx, par(k, -1);

  for (int i = 0; i < k; i++) {
    int j = lower_bound(tail.begin(), tail.end(), p[i]) - tail.begin();

    if (j > 0)
      par[i] = tail_idx[j - 1];

    if (j == (int)tail.size()) {
      tail.push_back(p[i]);
      tail_idx.push_back(i);
    } else {
      tail[j] = p[i];
      tail_idx[j] = i;
    }
  }

  vector<int> ans;
  if (!tail.empty()) {
    int cur = tail_idx.back();
    while (cur != -1) {
      ans.push_back(val[cur]);
      cur = par[cur];
    }
    reverse(ans.begin(), ans.end());
  }

  cout << ans.size() << '\n';
  for (int x : ans)
    cout << x << ' ';
  cout << '\n';
}
```

Subarray Sum Constraints

```
// cses/Additional Problems I/subarray_sum_constraints.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<vector<pair<int, ll>>> adj(n + 1);

  while (m--) {
    int l, r;
    ll s;
    cin >> l >> r >> s;
    --l;

    adj[l].push_back({r, s});
    adj[r].push_back({l, -s});
  }

  vector<ll> p(n + 1);
  vector<bool> vis(n + 1);

  for (int st = 0; st <= n; ++st) {
    if (vis[st])
      continue;

    queue<int> q;
    q.push(st);
    vis[st] = true;

    while (!q.empty()) {
      int u = q.front();
      q.pop();

      for (auto [v, w] : adj[u]) {
        ll want = p[u] + w;

        if (!vis[v]) {
          vis[v] = true;
          p[v] = want;
          q.push(v);
        } else if (p[v] != want) {
          cout << "NO\n";
          return 0;
        }
      }
    }
  }

  cout << "YES\n";
  for (int i = 1; i <= n; ++i)
    cout << p[i] - p[i - 1] << " \n"[i == n];
}
```

Subsets with Fixed Average

```
// cses/Additional Problems I/subsets_with_fixed_average.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, a;
  cin >> n >> a;

  vector<int> v(n);

  int pos = 0, neg = 0;

  for (int &x : v) {
    cin >> x;
    x -= a;

    if (x > 0)
      pos += x;
    else
      neg -= x;
  }

  sort(v.rbegin(), v.rend());

  int m = min(pos, neg);

  vector<int> dp(m + 1);
  dp[0] = 1;

  for (int x : v) {
    if (x > 0) {
      if (x > m)
        continue;

      for (int s = m; s >= x; --s) {
        dp[s] += dp[s - x];
        if (dp[s] >= MOD)
          dp[s] -= MOD;
      }
    } else if (x == 0) {
      for (int s = 0; s <= m; ++s) {
        dp[s] = 2LL * dp[s] % MOD;
      }
    } else {
      int d = -x;
```

```
      if (d > m)
        continue;

      for (int s = 0; s + d <= m; ++s) {
        dp[s] += dp[s + d];
        if (dp[s] >= MOD)
          dp[s] -= MOD;
      }
    }
  }

  cout << (dp[0] - 1 + MOD) % MOD << '\n';
}
```

Two Array Average

```
// cses/Additional Problems I/two_array_average.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<ll> a(n + 1), b(n + 1);

  for (int i = 1; i <= n; i++) {
    ll x;
    cin >> x;
    a[i] = a[i - 1] + x;
  }

  for (int i = 1; i <= n; i++) {
    ll x;
    cin >> x;
    b[i] = b[i - 1] + x;
  }

  long double lo = 0, hi = 1e9;

  for (int it = 0; it < 60; it++) {
    long double mid = (lo + hi) / 2;

    long double bestA = -1e30L;
    long double bestB = -1e30L;

    for (int i = 1; i <= n; i++) {
      bestA = max(bestA, (long double)a[i] - mid * i);
      bestB = max(bestB, (long double)b[i] - mid * i);
    }

    if (bestA + bestB >= 0)
      lo = mid;
    else
      hi = mid;
  }

  int x = 1, y = 1;
  long double bestA = -1e30L;
  long double bestB = -1e30L;

  for (int i = 1; i <= n; i++) {
    long double cur = (long double)a[i] - lo * i;
    if (cur > bestA) {
      bestA = cur;
      x = i;
    }
  }

  for (int i = 1; i <= n; i++) {
    long double cur = (long double)b[i] - lo * i;
    if (cur > bestB) {
      bestB = cur;
      y = i;
    }
  }

  cout << x << ' ' << y << '\n';
}
```

Water Containers Moves

```
// cses/Additional Problems I/water_containers_moves.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int a, b, x;
  cin >> a >> b >> x;

  if (x > a || x % gcd(a, b) != 0) {
    cout << -1 << '\n';
    return 0;
  }

  int W = b + 1;
  int N = (a + 1) * (b + 1);

  auto id = [&](int A, int B) { return A * W + B; };

  const ll INF = (1LL << 60);

  vector<ll> dist(N, INF);
  vector<int> par(N, -1);
  vector<char> how(N, -1);

  const string name[] = {"FILL A",  "FILL B",   "EMPTY A",
                         "EMPTY B", "MOVE A B", "MOVE B A"};

  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
      pq;

  int start = id(0, 0);
  dist[start] = 0;
  pq.push({0, start});

  int target = -1;

  while (!pq.empty()) {
    auto cur = pq.top();
    pq.pop();

    ll d = cur.first;
    int v = cur.second;

    if (d != dist[v])
      continue;

    int A = v / W;
    int B = v % W;

    if (A == x) {
      target = v;
      break;
    }

    auto relax = [&](int nA, int nB, int cost, int op) {
      if (cost == 0)
        return;

      int u = id(nA, nB);
      if (d + cost < dist[u]) {
        dist[u] = d + cost;
        par[u] = v;
        how[u] = op;
        pq.push({dist[u], u});
      }
    };

    relax(a, B, a - A, 0);
    relax(A, b, b - B, 1);
    relax(0, B, A, 2);
    relax(A, 0, B, 3);

    int t = min(A, b - B);
    relax(A - t, B + t, t, 4);

    t = min(B, a - A);
    relax(A + t, B - t, t, 5);
  }

  if (target == -1) {
    cout << -1 << '\n';
    return 0;
  }

  vector<int> ops;

  for (int v = target; v != start; v = par[v])
    ops.push_back(how[v]);

  reverse(ops.begin(), ops.end());

  cout << ops.size() << ' ' << dist[target] << '\n';

  for (int op : ops)
    cout << name[op] << '\n';
}
```

Water Containers Queries

```
// cses/Additional Problems I/water_containers_queries.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  while (n--) {
    ll a, b, x;
    cin >> a >> b >> x;

    cout << (x <= a && x % gcd(a, b) == 0 ? "YES\n" : "NO\n");
  }
}
```

Additional Problems II

Binary Subsequences

```cpp
// cses/Additional Problems II/binary_subsequences.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int steps(int a, int b) {
  int ans = -1;

  while (b) {
    ans += a / b;
    a %= b;
    swap(a, b);
  }

  return ans;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  int sum = n + 2;

  int best_len = INT_MAX;
  int best_a = 1, best_b = sum - 1;

  // (a,b) and (b,a) are symmetric.
  for (int a = 1; a * 2 <= sum; ++a) {
    int b = sum - a;

    if (gcd(a, b) != 1)
      continue;

    int len = steps(a, b);

    if (len < best_len) {
      best_len = len;
      best_a = a;
      best_b = b;
    }
  }

  int a = best_a, b = best_b;
  string ans;

  // Trace the subtractive Euclidean algorithm.
  while (a != b) {
    if (a > b) {
      int cnt = (a - 1) / b;
      ans.append(cnt, '1');
      a -= cnt * b;
    } else {
      int cnt = (b - 1) / a;
      ans.append(cnt, '0');
      b -= cnt * a;
    }
  }

  cout << ans << '\n';
}
```

Bouncing Ball Cycle

```cpp
// cses/Additional Problems II/bouncing_ball_cycle.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int t;
  cin >> t;

  while (t--) {
    ll n, m;
    cin >> n >> m;

    ll a = n - 1;
    ll b = m - 1;
    ll g = gcd(a, b);

    ll A = a / g;
    ll B = b / g;
    ll l = A * b; // lcm(a, b)

    ll steps = 2 * l;
    ll cells = l - (A - 1) * (B - 1) / 2 + 1;

    cout << steps << ' ' << cells << '\n';
  }
}
```

Bouncing Ball Steps

```cpp
// cses/Additional Problems II/bouncing_ball_steps.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll position(ll len, ll k) {
  ll d = len - 1;
  ll x = k % (2 * d);
  if (x > d)
    x = 2 * d - x;
  return x + 1;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int t;
  cin >> t;

  while (t--) {
    ll n, m, k;
    cin >> n >> m >> k;

    ll a = n - 1;
    ll b = m - 1;

    ll g = gcd(a, b);
    ll lcm = a / g * b;

    ll changes = k / a + k / b - k / lcm;

    cout << position(n, k) << ' ' << position(m, k) << ' ' << changes << '\n';
  }
}
```

GCD Subsets

```cpp
// cses/Additional Problems II/gcd_subsets.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<int> freq(n + 1);
  for (int i = 0, x; i < n; i++) {
    cin >> x;
    freq[x]++;
  }

  vector<ll> pw2(n + 1, 1);
  for (int i = 1; i <= n; i++)
    pw2[i] = pw2[i - 1] * 2 % MOD;

  vector<ll> ans(n + 1);

  for (int k = n; k >= 1; k--) {
    int cnt = 0;
    for (int j = k; j <= n; j += k)
      cnt += freq[j];

    ans[k] = pw2[cnt] - 1;

    for (int j = 2 * k; j <= n; j += k)
      ans[k] = (ans[k] - ans[j] + MOD) % MOD;
  }

  for (int k = 1; k <= n; k++)
    cout << ans[k] << " \n"[k == n];
}
```

Grid Coloring II

```cpp
// cses/Additional Problems II/grid_coloring_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  int N = n * m;
  vector<int> a(N);

  for (int i = 0; i < n; i++) {
    string s;
    cin >> s;
    for (int j = 0; j < m; j++)
      a[i * m + j] = s[j] - 'A';
  }

  int V = 2 * N;

  vector<int> head(V, -1), rhead(V, -1);
  vector<int> to, nxt, rto, rnxt;

  to.reserve(8 * N);
  nxt.reserve(8 * N);
  rto.reserve(8 * N);
  rnxt.reserve(8 * N);

  auto add_edge = [&](int u, int v) {
    to.push_back(v);
    nxt.push_back(head[u]);
    head[u] = (int)to.size() - 1;

    rto.push_back(u);
    rnxt.push_back(rhead[v]);
    rhead[v] = (int)rto.size() - 1;
  };

  auto add_adj = [&](int u, int v) {
    for (int c = 0; c < 3; c++) {
      if (c == a[u] || c == a[v])
        continue;

      int cu = (c == (a[u] + 1) % 3 ? 0 : 1);
      int cv = (c == (a[v] + 1) % 3 ? 0 : 1);

      int x = 2 * u + cu;
      int y = 2 * v + cv;

      // not (x and y)
      add_edge(x, y ^ 1);
      add_edge(y, x ^ 1);
    }
  };

  for (int i = 0; i < n; i++) {
```

```
  for (int j = 0; j < m; j++) {
    int u = i * m + j;
    if (i + 1 < n)
      add_adj(u, u + m);
    if (j + 1 < m)
      add_adj(u, u + 1);
  }
}

// Kosaraju, first pass
vector<char> vis(V);
vector<int> ptr = head, order, st;
order.reserve(V);
st.reserve(V);

for (int s = 0; s < V; s++) {
  if (vis[s])
    continue;

  vis[s] = 1;
  st.push_back(s);

  while (!st.empty()) {
    int v = st.back();
    int &e = ptr[v];

    while (e != -1 && vis[to[e]])
      e = nxt[e];

    if (e == -1) {
      order.push_back(v);
      st.pop_back();
    } else {
      int u = to[e];
      e = nxt[e];
      vis[u] = 1;
      st.push_back(u);
    }
  }
}

// Kosaraju, second pass
vector<int> comp(V, -1);
int cid = 0;

for (int k = V - 1; k >= 0; k--) {
  int s = order[k];
  if (comp[s] != -1)
    continue;

  comp[s] = cid;
  st.push_back(s);

  while (!st.empty()) {
    int v = st.back();
    st.pop_back();

    for (int e = rhead[v]; e != -1; e = rnxt[e]) {
      int u = rto[e];
      if (comp[u] == -1) {
        comp[u] = cid;
        st.push_back(u);
      }
    }
  }

  cid++;
}

for (int i = 0; i < N; i++) {
  if (comp[2 * i] == comp[2 * i + 1]) {
    cout << "IMPOSSIBLE\n";
    return 0;
  }
}

for (int i = 0; i < n; i++) {
  for (int j = 0; j < m; j++) {
    int v = i * m + j;
    int choice = comp[2 * v] > comp[2 * v + 1] ? 0 : 1;
    int color = (a[v] + 1 + choice) % 3;
    cout << char('A' + color);
  }
    cout << '\n';
  }
}
```

K Subset Sums I

```
// cses/Additional Problems II/k_subset_sums_I.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, k;
  cin >> n >> k;

  vector<ll> a(n);
  ll base = 0;

  for (ll &x : a) {
    cin >> x;
    if (x < 0)
      base += x;
    x = abs(x);
  }

  sort(a.begin(), a.end());

  // (subset sum, largest index used)
  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
      pq;

  cout << base;

  if (k == 1) {
    cout << '\n';
    return 0;
  }

  pq.push({a[0], 0});

  for (int cnt = 1; cnt < k; cnt++) {
    auto [sum, i] = pq.top();
    pq.pop();

    cout << ' ' << base + sum;

    if (i + 1 < n) {
      // Keep a[i], also add a[i+1].
      pq.push({sum + a[i + 1], i + 1});

      // Replace a[i] by a[i+1].
      pq.push({sum - a[i] + a[i + 1], i + 1});
    }
  }

  cout << '\n';
}
```

K Subset Sums II

```
// cses/Additional Problems II/k_subset_sums_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m, k;
  cin >> n >> m >> k;

  vector<ll> a(n);
  for (ll &x : a)
    cin >> x;
  sort(a.begin(), a.end());

  ll base = 0;
  for (int i = 0; i < m; i++)
    base += a[i];

  // (sum, p, x, y)
  // p = leftmost position differing from [0,1,...,m-1]
  // x = selected index at position p
  // y = selected index at position p+1, or n
  using T = tuple<ll, int, int, int>;
  priority_queue<T, vector<T>, greater<T>> pq;

  cout << base;
  int done = 1;

  if (m < n) {
    pq.push({base + a[m] - a[m - 1], m - 1, m, n});
  }

  while (done < k) {
    auto [sum, p, x, y] = pq.top();
    pq.pop();

    cout << ' ' << sum;
    done++;

    // Move the current changed index one step right.
    if (x + 1 < y) {
      pq.push({sum + a[x + 1] - a[x], p, x + 1, y});
    }

    // Make position p-1 the new leftmost changed position.
    if (p > 0) {
      pq.push({sum + a[p] - a[p - 1], p - 1, p, x});
    }
  }

  cout << '\n';
}
```

Knight Moves Queries

```
// cses/Additional Problems II/knight_moves_queries.cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

ll solve(ll x, ll y) {
  x--;
  y--;

  if (x < y)
    swap(x, y);

  if (x == 1 && y == 1)
    return 4;
  if (x == 1 && y == 0)
    return 3;
  if (x == 2 && y == 2)
    return 4;

  ll d = max((x + 1) / 2, (x + y + 2) / 3);

  if ((d + x + y) % 2)
    d++;
  return d;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  while (n--) {
    ll x, y;
    cin >> x >> y;
    cout << solve(x, y) << '\n';
  }
}
```

Maximum Building II

```cpp
// cses/Additional Problems II/maximum_building_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<string> g(n);
  for (auto &s : g)
    cin >> s;

  vector<vector<ll>> A(n + 2, vector<ll>(m + 2));
  vector<vector<ll>> B(n + 2, vector<ll>(m + 2));

  auto add = [&](int h, int l, int r, ll a, ll b) {
    if (h == 0 || l > r)
      return;

    A[1][l] += a;
    A[1][r + 1] -= a;
    A[h + 1][l] -= a;
    A[h + 1][r + 1] += a;

    B[1][l] += b;
    B[1][r + 1] -= b;
    B[h + 1][l] -= b;
    B[h + 1][r + 1] += b;
  };

  vector<int> h(m), prv(m), nxt(m), st;

  for (int row = 0; row < n; row++) {
    for (int j = 0; j < m; j++) {
      if (g[row][j] == '.')
        h[j]++;
      else
        h[j] = 0;
    }

    st.clear();
    for (int i = 0; i < m; i++) {
      while (!st.empty() && h[st.back()] >= h[i])
        st.pop_back();
      prv[i] = st.empty() ? -1 : st.back();
      st.push_back(i);
    }

    st.clear();
    for (int i = m - 1; i >= 0; i--) {
      while (!st.empty() && h[st.back()] > h[i])
        st.pop_back();
      nxt[i] = st.empty() ? m : st.back();
      st.push_back(i);
    }

    for (int i = 0; i < m; i++) {
      if (h[i] == 0)
        continue;

      int L = i - prv[i];
      int R = nxt[i] - i;

      int x = min(L, R);
      int y = max(L, R);
      int s = L + R;

      // Number of intervals of width w containing i:
      //
      // w                    for 1 <= w <= x
      // x                    for x < w <= y
      // L + R - w            for y < w <= L+R-1

      add(h[i], 1, x, 1, 0);
      add(h[i], x + 1, y, 0, x);
      add(h[i], y + 1, s - 1, -1, s);
    }
  }

  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
      A[i][j] += A[i - 1][j] + A[i][j - 1] - A[i - 1][j - 1];
      B[i][j] += B[i - 1][j] + B[i][j - 1] - B[i - 1][j - 1];

      cout << A[i][j] * j + B[i][j] << (j == m ? '\n' : ' ');
    }
  }
}
```

Minimum Cost Pairs

```cpp
// cses/Additional Problems II/minimum_cost_pairs.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<ll> x(n);
  for (ll &v : x)
    cin >> v;
  sort(x.begin(), x.end());

  const ll INF = (1LL << 62);

  // Edge i joins x[i-1] and x[i], for 1 <= i < n.
  // Edges 0 and n are infinite sentinels.
  vector<ll> w(n + 1, INF);
  vector<int> prv(n + 1), nxt(n + 1);
  vector<char> alive(n + 1, true);

  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
      pq;

  for (int i = 1; i < n; ++i) {
    w[i] = x[i] - x[i - 1];
    pq.push({w[i], i});
  }

  for (int i = 0; i <= n; ++i) {
    prv[i] = i - 1;
    nxt[i] = i + 1;
  }
  prv[0] = -1;
  nxt[n] = -1;

  ll ans = 0;

  for (int k = 1; k <= n / 2; ++k) {
    ll cost;
    int i;

    while (true) {
      tie(cost, i) = pq.top();
      pq.pop();

      if (alive[i] && cost == w[i])
        break;
    }

    ans += cost;
    cout << ans << (k == n / 2 ? '\n' : ' ');

    int l = prv[i];
    int r = nxt[i];

    // Regret choosing i:
    // replace it by choosing both neighboring edges.
    ll nw = INF;
    if (w[l] < INF / 2 && w[r] < INF / 2)
      nw = w[l] + w[r] - w[i];

    alive[l] = false;
    alive[r] = false;

    int ll = prv[l];
    int rr = nxt[r];

    prv[i] = ll;
    nxt[i] = rr;

    if (ll != -1)
      nxt[ll] = i;
    if (rr != -1)
      prv[rr] = i;

    w[i] = nw;
    if (nw < INF / 2)
      pq.push({nw, i});
  }
}
```

Removing Digits II

```cpp
// cses/Additional Problems II/removing_digits_II.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

unordered_map<ll, pair<ll, ll>> memo[10];

// mx = largest digit in the prefix outside n
// returns {number of steps, remainder after n is reduced to <= 0}
pair<ll, ll> solve(int mx, ll n) {
  if (n == 0)
    return {0, 0};
  if (n < 10)
    return {1, n - max<ll>(mx, n)};

  auto it = memo[mx].find(n);
  if (it != memo[mx].end())
    return it->second;

  ll p = 1;
  while (p <= n / 10)
    p *= 10;

  pair<ll, ll> ans;

  if (n % p == 0) {
    // Empty suffix: perform one greedy step directly.
    int d = max(mx, int(n / p));
    auto cur = solve(mx, n - d);
    ans = {cur.first + 1, cur.second};
  } else {
    int first = n / p;

    // Process the suffix while the leading digit stays unchanged.
    auto low = solve(max(mx, first), n % p);

    // The suffix may borrow from the prefix.
    auto high = solve(mx, n / p * p + low.second);

    ans = {low.first + high.first, high.second};
  }

  return memo[mx][n] = ans;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  ll n;
  cin >> n;

  cout << solve(0, n).first << '\n';
}
```

Replace with Difference

```cpp
// cses/Additional Problems II/replace_with_difference.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXS = 500001;

bitset<MAXS> dp[1001];

int main() {
  ios::sync_with_stdio(false);
```

```
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<int> a(n);
  int sum = 0;

  for (int &x : a) {
    cin >> x;
    sum += x;
  }

  if (sum % 2) {
    cout << -1 << '\n';
    return 0;
  }

  int target = sum / 2;

  dp[0][0] = 1;

  for (int i = 0; i < n; i++)
    dp[i + 1] = dp[i] | (dp[i] << a[i]);

  if (!dp[n][target]) {
    cout << -1 << '\n';
    return 0;
  }

  vector<int> A, B;

  int s = target;

  for (int i = n; i >= 1; i--) {
    if (dp[i - 1][s]) {
      B.push_back(a[i - 1]);
    } else {
      A.push_back(a[i - 1]);
      s -= a[i - 1];
    }
  }

  int i = 0, j = 0;
  int x = A[0], y = B[0];
  int zeros = 0;

  while (i < (int)A.size() && j < (int)B.size()) {
    cout << x << ' ' << y << '\n';

    if (x > y) {
      x -= y;
      j++;
      if (j < (int)B.size())
        y = B[j];
    } else if (y > x) {
      y -= x;
      i++;
      if (i < (int)A.size())
        x = A[i];
    } else {
      zeros++;
      i++;
      j++;

      if (i < (int)A.size())
        x = A[i];
      if (j < (int)B.size())
        y = B[j];
    }
  }

  for (int k = 1; k < zeros; k++)
    cout << "0 0\n";
}
```

Same Sum Subsets

```
// cses/Additional Problems II/same_sum_subsets.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<ll> x(n);
  for (ll &v : x)
    cin >> v;

  int m = n / 2;

  auto gen = [&](int l, int r) {
    int k = r - l, N = 1 << k;
    vector<pair<ll, int>> v(N);

    for (int mask = 1; mask < N; ++mask) {
      int b = __builtin_ctz(mask);
      int pm = mask ^ (1 << b);
      v[mask] = {v[pm].first + x[l + b], mask};
    }

    sort(v.begin(), v.end());
    return v;
  };

  auto A = gen(0, m);
  auto B = gen(m, n);

  vector<ll> sa(A.size()), sb(B.size());

  for (int i = 0; i < (int)A.size(); ++i)
    sa[i] = A[i].first;
  for (int i = 0; i < (int)B.size(); ++i)
    sb[i] = B[i].first;

  // Number of non-empty subsets with sum <= s.
  auto count_le = [&](ll s) {
    ll cnt = 0;
    int j = (int)sb.size() - 1;

    for (ll a : sa) {
      while (j >= 0 && a + sb[j] > s)
        --j;

      if (j < 0)
        break;
      cnt += j + 1;
    }

    return cnt - 1; // remove empty subset
  };

  // Find adjacent lo, hi such that
  // count_le(lo) <= lo and count_le(hi) > hi.
  ll lo = 0;
  ll hi = accumulate(x.begin(), x.end(), 0LL);

  while (hi - lo > 1) {
    ll mid = (lo + hi) / 2;

    if (count_le(mid) > mid)
      hi = mid;
    else
      lo = mid;
  }

  // At least two subsets have sum exactly hi.
  ll target = hi;

  ull p = 0, q = 0;
  bool have = false, found = false;

  int i = 0;
  int j = (int)B.size() - 1;

  while (i < (int)A.size() && j >= 0) {
    ll s = A[i].first + B[j].first;

    if (s < target) {
      ++i;
    } else if (s > target) {
      --j;
    } else {
      ll a = A[i].first;
      ll b = B[j].first;

      int ni = i + 1;
      int nj = j - 1;

      while (ni < (int)A.size() && A[ni].first == a)
        ++ni;
      while (nj >= 0 && B[nj].first == b)
        --nj;

      auto mask = [&](int u, int v) {
        return (ull)A[u].second | ((ull)B[v].second << m);
      };

      ull cur = mask(i, j);

      if (!have) {
        p = cur;
        have = true;

        if (ni - i >= 2) {
          q = mask(i + 1, j);
          found = true;
          break;
        }

        if (j - nj >= 2) {
          q = mask(i, j - 1);
          found = true;
          break;
        }
      } else {
        q = cur;
        found = true;
        break;
      }

      i = ni;
      j = nj;
    }
  }

  if (!found) {
    cout << "IMPOSSIBLE\n";
    return 0;
  }

  // Cancel their intersection.
  ull common = p & q;
  p ^= common;
  q ^= common;

  auto print = [&](ull mask) {
    cout << __builtin_popcountll(mask) << '\n';

    for (int i = 0; i < n; ++i)
      if (mask >> i & 1ULL)
        cout << x[i] << ' ';

    cout << '\n';
  };

  print(p);
  print(q);
}
```

Stick Difference

```
// cses/Additional Problems II/stick_difference.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m;
  cin >> n >> m;

  vector<ll> a(n);
```

```
  for (ll &x : a)
    cin >> x;

  // hi[k] = best possible maximum piece after k cuts.
  // lo[k] = minimum piece in the greedy construction for hi[k].
  vector<ll> hi(m + 1), lo(m + 1), diff(m + 1);
  vector<int> cnt(n, 1);

  priority_queue<pair<ll, int>> pq;
  for (int i = 0; i < n; ++i)
    pq.push({a[i], i});

  hi[0] = pq.top().first;
  lo[0] = *min_element(a.begin(), a.end());
  diff[0] = hi[0] - lo[0];

  for (int k = 1; k <= m; ++k) {
    int i = pq.top().second;
    pq.pop();

    ++cnt[i];

    ll mx = (a[i] + cnt[i] - 1) / cnt[i];
    ll mn = a[i] / cnt[i];

    pq.push({mx, i});

    hi[k] = pq.top().first;
    lo[k] = min(lo[k - 1], mn);
    diff[k] = hi[k] - lo[k];
  }

  // best_min = largest possible minimum piece after k cuts.
  fill(cnt.begin(), cnt.end(), 1);

  priority_queue<pair<ll, int>> next_min;
  for (int i = 0; i < n; ++i)
    next_min.push({a[i] / 2, i});

  ll best_min = lo[0];

  // p = last j with lo[j] >= best_min.
  // q maintains min(diff[j]) for j in (p, k].
  int p = 0;
  deque<int> q;

  for (int k = 1; k <= m; ++k) {
    auto [mn, i] = next_min.top();
    next_min.pop();

    best_min = min(best_min, mn);

    ++cnt[i];
    next_min.push({a[i] / (cnt[i] + 1), i});

    while (!q.empty() && diff[q.back()] >= diff[k])
      q.pop_back();
    q.push_back(k);

    while (p < k && lo[p + 1] >= best_min)
      ++p;

    while (!q.empty() && q.front() <= p)
      q.pop_front();

    ll ans = hi[p] - best_min;

    if (!q.empty())
      ans = min(ans, diff[q.front()]);

    cout << ans << (k == m ? '\n' : ' ');
  }
}
```

Two Stacks Sorting

```
// cses/Additional Problems II/two_stacks_sorting.cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct DSU {
  vector<int> p, sz, xr;
  vector<set<int>> rights;

  DSU(int n) : p(n), sz(n, 1), xr(n), rights(n) { iota(p.begin(), p.end(), 0); }

  pair<int, int> find(int x) {
    if (p[x] == x)
      return {x, 0};
    auto q = find(p[x]);
    xr[x] ^= q.second;
    p[x] = q.first;
    return {p[x], xr[x]};
  }

  // Enforce color[a] != color[b].
  bool unite(int a, int b) {
    auto A = find(a);
    auto B = find(b);

    if (A.first == B.first)
      return (A.second ^ B.second) == 1;

    if (sz[A.first] < sz[B.first])
      swap(A, B);

    int ra = A.first, rb = B.first;
    p[rb] = ra;
    xr[rb] = A.second ^ B.second ^ 1;
    sz[ra] += sz[rb];

    if (rights[ra].size() < rights[rb].size())
      rights[ra].swap(rights[rb]);

    rights[ra].insert(rights[rb].begin(), rights[rb].end());
    rights[rb].clear();

    return true;
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n;
  cin >> n;

  vector<int> a(n), pos(n), seen(n);
  vector<int> L(n), R(n);

  for (int &x : a) {
    cin >> x;
    --x;
  }

  // Build intervals with distinct endpoints.
  vector<pair<int, int>> event(2 * n); // {index, 0=open / 1=close}

  int timer = 0, need = 0;

  for (int i = 0; i < n; ++i) {
    pos[a[i]] = i;
    seen[a[i]] = 1;

    L[i] = timer;
    event[timer++] = {i, 0};

    while (need < n && seen[need]) {
      int j = pos[need++];
      R[j] = timer;
      event[timer++] = {j, 1};
    }
  }

  DSU dsu(n);

  for (int i = 0; i < n; ++i)
    dsu.rights[i].insert(R[i]);

  // One active representative per DSU component.
  set<pair<int, int>> active; // {right endpoint, interval index}

  for (int t = 0; t < 2 * n; ++t) {
    int i = event[t].first;
    int type = event[t].second;

    if (type == 0) {
      int bestR = R[i];
      int bestI = i;

      // These intervals cross interval i.
      while (!active.empty() && active.begin()->first < R[i]) {
        auto cur = *active.begin();
        active.erase(active.begin());

        if (cur.first < bestR) {
          bestR = cur.first;
          bestI = cur.second;
        }

        if (!dsu.unite(i, cur.second)) {
          cout << "IMPOSSIBLE\n";
          return 0;
        }
      }

      active.insert({bestR, bestI});
    } else {
      auto it = active.find({R[i], i});

      // i may not currently be its component's representative.
      if (it == active.end())
        continue;

      active.erase(it);

      int root = dsu.find(i).first;
      auto nxt = dsu.rights[root].upper_bound(R[i]);

      if (nxt != dsu.rights[root].end()) {
        int r = *nxt;
        int j = event[r].first;
        active.insert({r, j});
      }
    }
  }

  vector<int> color(n);

  for (int i = 0; i < n; ++i)
    color[i] = dsu.find(i).second;

  // Verify by greedily popping whenever possible.
  stack<int> st[2];
  need = 0;

  for (int i = 0; i < n; ++i) {
    st[color[i]].push(a[i]);

    while (true) {
      if (!st[0].empty() && st[0].top() == need) {
        st[0].pop();
        ++need;
      } else if (!st[1].empty() && st[1].top() == need) {
        st[1].pop();
        ++need;
      } else {
        break;
      }
    }
  }

  if (need != n) {
    cout << "IMPOSSIBLE\n";
    return 0;
  }

  for (int i = 0; i < n; ++i)
    cout << color[i] + 1 << " \n"[i + 1 == n];
}
```